On the Diagram of Thought

Yifan Zhang[1] **Yang Yuan**[1,2] **Andrew Chi-Chih Yao**[1,2†]

[1]IIIS, Tsinghua University [2]Shanghai Qi Zhi Institute

Abstract

Large Language Models (LLMs) excel at many tasks but often falter on complex problems that require structured, multi-step reasoning. We introduce the Diagram of Thought (DoT), a framework that enables a single LLM to build and navigate a mental map of its reasoning. Instead of thinking in a straight line, the model constructs a dynamic diagram of ideas, where it can propose different lines of thought, critique its own steps, and synthesize validated insights into a final conclusion. This process is controller-light: it does not require an external search algorithm or planner, but it does use a deterministic online validator for grammar-constrained typed traces, register constraints, and optional solver checks. To clarify the reliability target of this process, we ground DoT in a mathematical framework from category theory. We interpret accepted typed reasoning records as diagrams in a slice topos and model synthesis of the selected proposer subdiagram as a finite limit. In the predicate fragment, this same object is equivalently a variance-reversed colimit in the opposite information order. The resulting formalism gives an auditable, step-by-step trace of the LLM's typed reasoning and separates semantic guarantees for the typed subtrace from unconstrained natural-language text and uncertified operational edges.

Project Page: https://github.com/diagram-of-thought/diagram-of-thought

1 Introduction

Large Language Models (LLMs) (Brown et al., 2020; Touvron et al., 2023) have exhibited remarkable proficiency across a spectrum of natural language tasks. However, achieving robust performance on complex reasoning problems that necessitate structured exploration, iterative refinement, backtracking, and self-correction remains a formidable challenge (Huang and Chang, 2022). Initial prompting strategies, such as Chain-of-Thought (CoT) (Wei et al., 2022), encourage step-by-step reasoning by eliciting intermediate steps. While beneficial, the inherent linearity of CoT struggles to capture the dynamic, non-sequential nature of sophisticated problem-solving, which often involves generating parallel hypotheses, critical evaluation, and synthesis, processes ill-suited to a strictly linear progression.

Recognizing these limitations, subsequent research has explored more complex reasoning structures. Frameworks like Tree-of-Thought (ToT) (Yao et al., 2023) utilize tree structures to manage multiple reasoning paths, while Graph-of-Thought (GoT) (Besta et al., 2024) generalizes this to arbitrary graphs. Other approaches, such as Cumulative Reasoning (CR) (Zhang et al., 2023), leverage multi-agent paradigms with specialized roles. Despite their advancements, these methods frequently require external controllers or multi-component pipelines. In contrast, DoT retains a single-model setting while making the operational constraints explicit and checkable via grammar-constrained masking with register constraints and typed validation.

In this paper, we introduce the Diagram of Thought (DoT) framework, which internalizes complex, iterative reasoning within a single auto-regressive language model, guided by learned role tokens and a typed serialization enforced by an online validator with finite control over record kinds and register/solver checks. DoT conceptualizes the reasoning process as the construction of a Directed Acyclic Graph (DAG), as in Figure 1. Nodes represent propositions, critiques, refinements, or verified statements, while edges capture logical or procedural dependencies. The model explores alternatives, responds to critiques, and consolidates validated content toward a conclusion.

A cornerstone of the DoT framework is its operationalization through role-specific tokens (e.g., `<proposer>`, `<critic>`, `<summarizer>`). By learning to predict and condition on these tokens, the model transitions between cognitive roles, generating hypotheses, evaluating them, refining based on feedback, and synthesizing results, within standard auto-regressive decoding. Tokens are training/decoding aids; the recorded `@node` role is the validator's source of truth. This unifies the entire reasoning process inside one model while keeping the interface auditable. All formal guarantees below are scoped to the typed subtrace accepted by V; untyped/free text is semantically inert for Φ and may diverge in natural language from the typed content.

Crucially, to ensure logical consistency and provide a principled aggregation mechanism, we establish a mathematical foundation for DoT using Topos Theory (MacLane and Moerdijk, 2012; Johnstone, 2002; Lambek and Scott, 1988). This framework allows us to model reasoning steps as formal objects and their synthesis as a universal construction. Specifically, we fix a presheaf topos $\mathcal{E} = \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ and a semantic object $S \in \mathcal{E}$. A typed proposer node is interpreted as an object of the slice $(\mathcal{E}/S)$ (a map $X \to S$); in the predicate instantiation used for most intuitions, this map is a monomorphism $P \hookrightarrow S$ and thus corresponds to a subobject. Synthesis by the `<summarizer>` role is modeled as a finite limit of the selected proposer diagram in the slice. In the predicate case this is a meet in $\mathrm{Sub}(S)$, equivalently the corresponding colimit after reversing variance into the opposite information order $\mathrm{Pred}(S) = \mathrm{Sub}(S)^{\mathrm{op}}$. Operational dependency edges contribute to provenance; they become semantic arrows only when accompanied by accepted typed certificates. Reflection along a Lawvere–Tierney topology captures validation; for a finite family of already validated (c-closed) predicates, the outer closure is redundant because the associated closure is a nucleus. For general slice objects that are not monomorphisms, validation is expressed by applying the induced slice sheafification functor; the phrase "c-closed" is reserved for subobjects.

Our main contributions are therefore:

1. We propose the Diagram of Thought (DoT), a single-model framework for DAG-structured proposals, critiques, and summaries, with a deterministic record serialization whose regular core is supplemented by register and solver checks, together with a finite-control online validator that enables auditable extraction.
2. We develop a categorical semantics for DoT in the slice topos $(\mathcal{E}/S)$ and show that synthesis of a selected finite validated proposer diagram is a finite limit in the slice. In the predicate/mono fragment this is the meet of the selected validated subobjects, equivalently a variance-reversed information-order colimit; in the general-arrow fragment, left-exact slice sheafification reflects this finite limit into the validated/sheaf slice.
3. We define a total and deterministic extraction map from LLM-generated traces to formal diagrams, enabled by a well-formedness discipline (BNF grammar, operational rules) and constrained decoding, ensuring that typed reasoning traces are auditable and structurally sound.

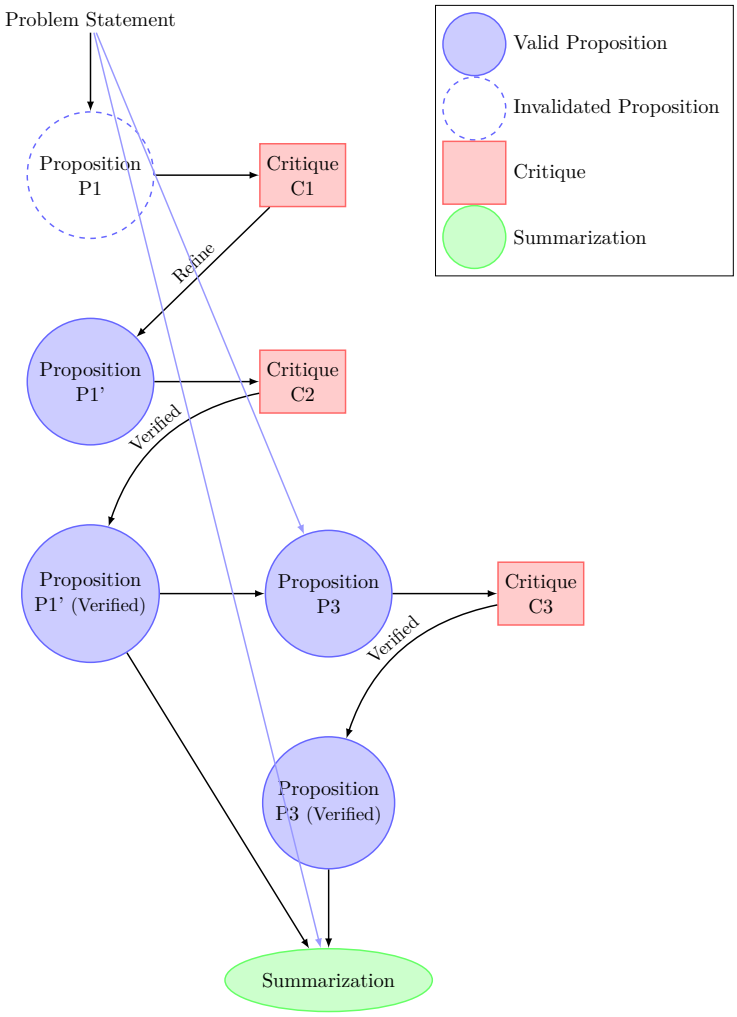

Figure 1 High-level illustration of the Diagram of Thought (DoT) process. Edges encode dependencies from earlier to later nodes: a critic depends on the proposition it evaluates (proposer $\rightarrow$ critic), and the summarizer depends on validated propositions. A single LLM generates the DAG representing proposing (circles), critiquing (rectangles), repairing/verification, and synthesis (ellipse).

2 Related Work

The pursuit of robust reasoning within Large Language Models (LLMs) has driven considerable research beyond basic input-output functionality. Initial breakthroughs like Chain-of-Thought (CoT) prompting (Wei et al., 2022; Kojima et al., 2022) demonstrated that eliciting intermediate reasoning steps significantly improves performance on complex tasks. CoT effectively linearizes reasoning, enhancing transparency but suffering from rigidity; its sequential nature hinders exploration of alternatives or recovery from early errors without restarting. Methods like Self-consistency (Wang et al., 2022) mitigate this by sampling multiple reasoning paths and selecting the majority answer, implicitly acknowledging path diversity but lacking explicit refinement mechanisms.

Recognizing the constraints of linearity, subsequent work explored more complex structures. Tree-of-Thought (ToT) (Yao et al., 2023) introduced tree structures where nodes represent partial

solutions and edges denote reasoning operators. ToT utilizes search algorithms (e.g., BFS, DFS) guided by heuristic evaluations (often LLM-based) to explore possibilities, enabling systematic search and backtracking. However, ToT generally necessitates an external controller for search management and pruning. Graph-of-Thought (GoT) (Besta et al., 2024) extends this to arbitrary graphs, allowing for more intricate dependency modeling, such as merging reasoning paths, but often requiring more sophisticated external graph management systems.

Collaborative and iterative refinement approaches offer another perspective. Cumulative Reasoning (CR) (Zhang et al., 2023) employs multiple LLM instances (or prompts) assigned specific roles (e.g., proposer, verifier), interacting iteratively. While modular, this introduces coordination overhead. Self-Refine (Madaan et al., 2023) focuses on iterative improvement where an LLM critiques and refines its own output, though typically applied to the entire output rather than intermediate reasoning steps within a structured process.

From a foundational perspective, Yuan (2023) uses category theory to analyze the inherent capabilities and limitations of LLMs. This work proves that prompt-based tuning is restricted to "representable" tasks within the pretext task category, potentially explaining the limitations of simpler methods like CoT. Conversely, the theory suggests fine-tuning offers broader potential, theoretically enabling a sufficiently powerful model to solve any task within that category given adequate resources.

Diagram of Thought (DoT) builds upon these diverse approaches while offering key distinctions. Like ToT and GoT, DoT utilizes non-linear structures (DAGs) for reasoning. However, it distinctively internalizes the graph construction and navigation within a *single* auto-regressive model via role tokens, minimizing external control dependencies. This contrasts with the external orchestration often required by ToT and GoT. DoT employs explicit cognitive roles (propose, critique, summarize), similar to CR, but integrates them seamlessly within one model through conditional generation, avoiding multi-agent coordination complexities. The use of rich natural language critiques potentially offers more nuanced feedback than the simple heuristic scores sometimes used in ToT. Importantly, by grounding the typed reasoning trace in Topos Theory, DoT aims for a level of formal rigor and consistency criteria that distinguishes it from purely heuristic methods, resonating with the structural analysis provided by works like Yuan (2023). DoT thus presents a unified, self-contained, interpretable, and formally grounded approach to advance complex reasoning in LLMs.

3 The Diagram of Thought Framework

In this section, we formally define the Diagram of Thought (DoT) framework. DoT operationalizes complex, iterative reasoning as the dynamic construction and traversal of a Directed Acyclic Graph (DAG) $G = (V, E)$ entirely within a single auto-regressive language model, LM. This internal graph structure allows the model to manage parallel lines of thought, critique intermediate steps, repair ideas based on feedback, and synthesize validated conclusions.

Definition 3.1 (DoT Graph Components)**.** The DoT graph $G = (V, E)$ is composed of:

- **Nodes** $v \in V$: Each node represents a semantic unit or reasoning step. Every node v is associated with:
  - A specific role $\text{role}(v) \in R = \{\text{Problem, Proposer, Critic, Summarizer}\}$.
  - Textual content $\text{content}(v)$, generated by the LLM LM while assuming the role $\text{role}(v)$.

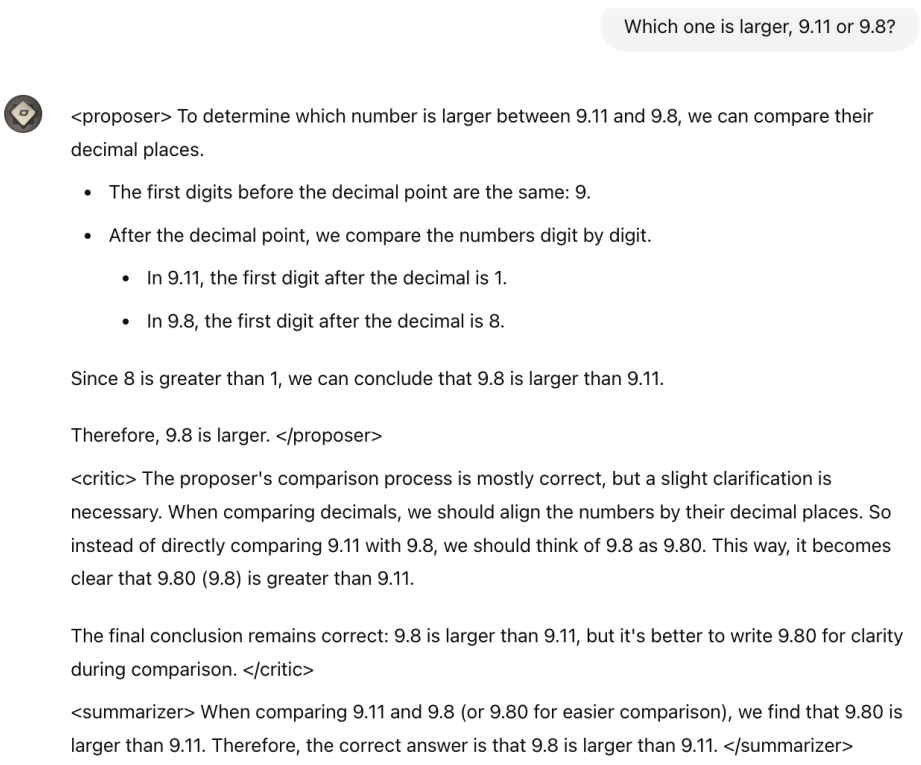

Figure 2 Illustrative example: Applying DoT reasoning steps to compare numerical values. Critiques might identify incorrect digit comparisons.

  - Optionally, an internal state $\text{state}(v) \in \{\text{active, validated, invalidated, initial}\}$. For example, a 'Proposer' node might start as 'active', become 'validated' after a positive critique, or 'invalidated' after a negative critique.
- **Edges** $(u, v) \in E$: A directed edge from node u to node v encodes that v *depends on* u. We standardize that, when emitting node v (with fresh ID v), all edges in its block have $dst{=}\, v$ and $src{<}\, v$. This covers:
  - Logical dependency (a new proposition depends on earlier premises). A node may depend on multiple sources, indicated by multiple edge records in its emission block.
  - Procedural dependency (a critic depends on the proposition it evaluates: proposer $\rightarrow$ critic).
  - Contextual dependency (a summarizer depends on the validated nodes it uses).

  The operational provenance graph is acyclic by construction since every operational edge targets the current node and all sources are previously emitted nodes. This acyclicity statement concerns the provenance DAG only: the extracted semantic index category may contain arrows opposite to refinement provenance, isomorphisms, or cycles induced by certified equivalence records.

The construction of this graph is implicitly managed by the LLM's standard auto-regressive generation process, strategically guided by special role tokens and constrained by the validator whenever typed serialization is enabled.

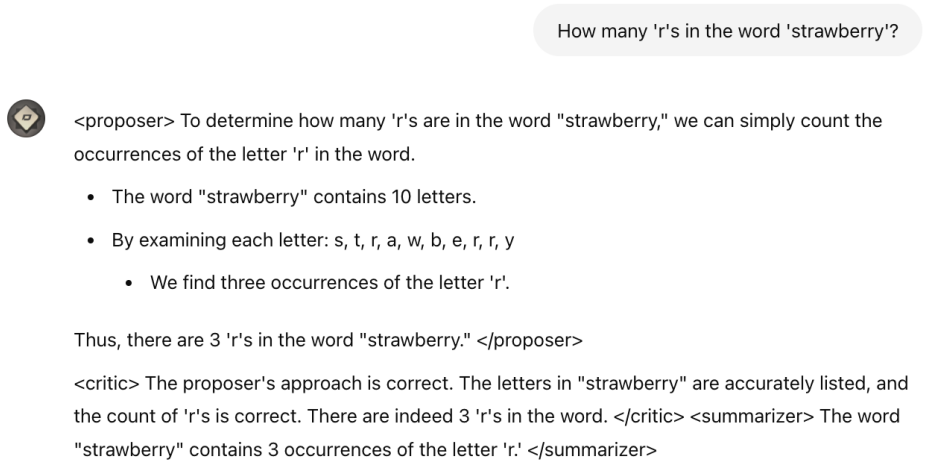

Figure 3 Illustrative example: A character-counting task where intermediate steps (identifying 'r's) and potential critiques (missed counts, double counts) could form a DoT graph.

3.1 Roles, Generation, and Iterative Reasoning

A core mechanism of DoT involves augmenting the LLM's vocabulary $\mathcal{V}$ with a distinct set of role-specific tokens:

$$T_{\text{roles}} = \{\texttt{<problem>}, \texttt{<proposer>}, \texttt{<critic>}, \texttt{<summarizer>}\}.$$

Let $\mathcal{V}' = \mathcal{V} \cup T_{\text{roles}}$ be the augmented vocabulary. The LLM LM operates by predicting the next token $w_t \in \mathcal{V}'$ based on the preceding sequence (history) $H_{t-1} = w_1, \ldots, w_{t-1}$:

$$\Pr_{\text{LM}_\theta}(w_t \mid H_{t-1}),$$

where θ denotes the model parameters. The generated sequence $H_T = w_1, \ldots, w_T$ represents a serialized traversal and construction of the DoT graph G.

The role tokens function as control signals, prompting the LLM to adopt a specific cognitive function for the subsequent text generation, thereby determining the role and content of the next node(s) in the graph:

- `<problem>`: Typically precedes the initial problem statement $\mathcal{P}$. This establishes the root node $v_{\text{start}} \in V$ with $\text{role}(v_{\text{start}}) = \text{Problem}$ and $\text{content}(v_{\text{start}}) = \mathcal{P}$. Its state is $\text{state}(v_{\text{start}}) = \text{initial}$.
- `<proposer>`: Signals the LLM to generate a hypothesis, intermediate reasoning step, or potential solution fragment P_i. This creates a new node v_{P_i} with $\text{role}(v_{P_i}) = \text{Proposer}$ and $\text{content}(v_{P_i}) = P_i$. All dependencies are declared explicitly via serialized `@edge` records defined in Section 3.2. The new node typically starts with $\text{state}(v_{P_i}) = \text{active}$.
- `<critic>`: Instructs the LLM to evaluate one or more preceding 'Proposer' nodes. The LLM generates a critique C_j, assessing validity, identifying flaws, or suggesting improvements. This creates a node v_{C_j} with $\text{role}(v_{C_j}) = \text{Critic}$ and explicit dependency edges from the propositions to the critic. We adopt a monotone validation discipline:

  - If the critique validates a proposition, its state transitions to 'validated'. This state is absorbing under the single-assignment discipline.
  - If the critique identifies flaws, its state transitions to 'invalidated'. This renders the proposition ineligible for summarization. The reasoning path is expected to continue from the critique node to generate a repair or alternative proposition.
- `<summarizer>`: Prompts the LLM to synthesize a final answer or consolidated conclusion. The model is trained to condition its summary generation on validated propositions explicitly selected by `@edge kind=use` records into the summarizer node, which are accessible through the serialized history H_{t-1}. It performs a conceptual aggregation respecting the declared dependencies and generates the summary text S. This creates a final node v_S with role(v_S) = Summarizer and explicit `@edge` records from the validated propositions that contributed to the summary. A full-run summary may select all validated propositions by convention. Generation often terminates after this step.

The LLM learns to predict appropriate role token transitions based on the entire preceding history H_{t-1}, effectively learning to navigate and structure the reasoning process. For instance, after generating a proposition via `<proposer>`, the model learns it is often appropriate to predict `<critic>`. Following a critical critique (`<critic>` leading to state 'invalidated'), the model might predict `<proposer>` again to generate a repair, or explore an alternative branch.

The DoT reasoning process unfolds as the LLM generates a serialized representation of the DAG G:

1. **Initialization**: The process begins with the problem statement, formatted using the typed serialization from Section 3.2 (e.g., `@node id=1 role=problem ...`). This defines the root node v_1.
2. **Proposal**: The LLM predicts `<proposer>` and generates the text for a proposition P_2, along with its node definition (`@node id=2 role=proposer`) and an edge from a prior node (`@edge src=1 dst=2 kind=use`). This adds node v_2 (state: active) to G.
3. **Critique**: The LLM predicts `<critic>`, generates a critique C_3 for P_2, and emits the corresponding node and edge records (`@node id=3 role=critic`, `@edge src=2 dst=3 kind=critique`). It also emits a status record (`@status target=2 mark=validated|invalidated`) that updates the state of v_2.
4. **Continuation (Branching/Repair/Exploration)**: Based on the history (including the state of v_2), the LLM predicts the next role token:
   - If v_2 was validated: The LLM might predict `<proposer>` to generate a new proposition P_4 building upon v_2 (adding node v_4 and an `@edge` from v_2).
   - If v_2 was invalidated by C_3: The LLM might predict `<proposer>` to generate a repaired proposition P_4 that addresses the critique, with a procedural edge from v_3 (`@edge src=3 dst=4 kind=refine`). Such a repair edge records provenance; it induces a semantic entailment arrow only if accompanied by a typed certificate such as `@entails`. Alternatively, it might backtrack to generate an alternative proposition P_5 stemming from the root node v_1.
5. **Iteration**: Steps 2–4 repeat, progressively extending the operational DAG. The requirement that operational edge destinations must have higher IDs than their sources syntactically enforces acyclicity of provenance.
6. **Summarization**: Eventually, the LLM predicts `<summarizer>`. It is trained to generate a summary by synthesizing information from selected validated nodes, adding a summary node,

and explicit `@edge` records from the nodes it used.
This process yields a structured, interpretable trace of reasoning, captured within a single, self-contained generated sequence.

3.2 Typed Serialization, Validation, and Extraction

To ground the reasoning process and ensure auditability, we introduce a disciplined, typed serialization format. Natural language content is interleaved with structured records (prefixed with '@') that define the DAG's nodes, edges, and states. For example, `@node id=3 role=critic` creates a critic node, and `@edge src=2 dst=3 kind=critique` establishes its dependency on a previous proposition. Node IDs are strictly increasing, guaranteeing acyclicity of the operational provenance DAG by construction.

During inference, a lightweight, controller-light validator enforces this structure. It uses grammar- and register-constrained masking to ensure the LLM only generates syntactically valid records (e.g., an edge's source must be a previously defined node) and, where typed blocks are present, calls the selected solver checks for semantic certificates. This process is deterministic and avoids external search algorithms or planners. A deterministic extraction map, Φ, converts any well-formed typed trace into a formal syntactic diagram, and then into a semantic diagram after a typed interpretation is fixed, enabling the categorical semantics described in Section 4. The full grammar, operational semantics, and validation rules are detailed in Appendix B.

3.3 Training and Controller-Light Inference

Training: The DoT capability is instilled in the LLM LM through fine-tuning on datasets formatted according to the DoT structure. Such data consists of sequences $H = w_1, \ldots, w_T$ containing appropriately interleaved role tokens (T_{roles}) and natural language text segments, representing valid, coherent reasoning DAGs. Potential data sources include:

- Curated examples derived from human step-by-step reasoning traces, augmented with role tokens and serialized records.
- Synthetically generated examples from structured problem-solving processes (e.g., program execution traces, mathematical proofs), formatted using the typed serialization from Section 3.2.
- Bootstrapped data generated by an initial version of a DoT model, potentially filtered or refined based on correctness or coherence metrics.

The training objective is the standard auto-regressive language modeling loss (e.g., cross-entropy) applied over the entire sequence, including role tokens and content tokens:

$$\mathcal{L}(\theta) = -\frac{1}{|H|} \sum_{t=1}^{|H|} \log \Pr_{\text{LM}_\theta}(w_t \mid w_1, \ldots, w_{t-1}).$$

This objective trains the model LM to simultaneously learn the reasoning patterns associated with each role (proposing, critiquing, summarizing) and the appropriate transitions between these roles based on the context, thereby internalizing the ability to construct and navigate the DoT graph structure.

Inference: To solve a new problem $\mathcal{P}$ using DoT, inference proceeds as follows:

1. Initialize the generation history H with the serialized problem statement, e.g., `@node id=1 role=problem ....`

2. Perform auto-regressive generation using the trained LLM LM. At each step t, sample or select the next token $w_t \sim \text{LM}(H_{t-1})$ using a chosen decoding strategy (e.g., greedy decoding, nucleus sampling) and the well-formedness constraints from Section 3.2.
3. Append the generated token w_t to the history: $H_t = H_{t-1} \oplus w_t$.
4. Repeat steps 2 and 3 until a termination condition is met. Common conditions include:
   - Generation of the `<summarizer>` token and its subsequent content, followed by a special end token.
   - Reaching a predefined maximum sequence length or generation budget.
   - Generation of a specific end-of-sequence token.

The final output is typically the textual content associated with the `<summarizer>` node, although the complete generated sequence H provides the full reasoning trace (the serialized DoT graph) for interpretability. Notably, this entire process is self-contained within the single LLM LM; no external graph management system or search/planning controller is required during inference, beyond a deterministic syntax validator and, when typed blocks are present, a decidable entailment check for the chosen typed fragment.

4 Topos-Theoretic Formalization of DoT

Order convention. We write $\text{Sub}(S)$ for the Heyting algebra of subobjects of S, ordered by inclusion ($P \leq Q$ iff $P \hookrightarrow Q$). Under this order, more informative / more constrained propositions correspond to smaller subobjects. This order-theoretic presentation applies to the predicate/mono instantiation in which proposer nodes are interpreted as monomorphisms into S. To align "more information" with "larger" elements, we work in the opposite poset

$$\text{Pred}(S) := \text{Sub}(S)^{\text{op}},$$

so that $P \preceq Q$ in $\text{Pred}(S)$ iff $Q \leq P$ in $\text{Sub}(S)$.

For a finite discrete family, the colimit in $\text{Pred}(S)$ is its join; translated back to $\text{Sub}(S)$ this is a meet (intersection):

$$\text{colim}_{\text{Pred}(S)}\{P_v\}_{v\in V} \;=\; \bigvee\nolimits_{\text{Pred}(S)}\{P_v\}_{v\in V} \;=\; \bigwedge\nolimits_{v\in V} P_v \quad \text{(computed in Sub}(S)\text{)}.$$

The empty meet is S, the terminal object of the slice $(\mathcal{E}/S)$. For an actual semantic diagram $D : \mathcal{J} \to \text{Sub}(S)$, the variance is explicit: the corresponding information-order diagram is $D^{\text{op}} : \mathcal{J}^{\text{op}} \to \text{Pred}(S) = \text{Sub}(S)^{\text{op}}$, and

$$\text{colim}_{\text{Pred}(S)}\, D^{\text{op}} \;\cong\; \lim_{\text{Sub}(S)} D.$$

Equivalently (and this is the formulation we use in the general-arrow setting), conjunction-like synthesis is a finite limit in the slice $(\mathcal{E}/S)$: for subobjects, finite limits are pullbacks and compute intersections. We keep the posetal fragment explicit (Assumption (A4)); the general-arrow case is handled via slice sheafification together with finite limits (Cor. 4.6).

General slice instantiation. When proposer nodes are interpreted as general objects $X \to S$ in $(\mathcal{E}/S)$ (not necessarily monos), the poset $\text{Sub}(S)$ no longer captures the whole semantics. Synthesis is then the finite limit of the extracted selected diagram in $(\mathcal{E}/S)$, not merely a meet

of its objects; validation reflects this finite limit into the sheaf slice via the induced left-exact functor $a_j^{/S} : (\mathcal{E}/S) \to (\mathrm{Sh}_j(\mathcal{E})/a_j S)$. The predicate/mono equations are recovered by restricting to monomorphisms and comparing the sheafified result back over S by pullback along the unit $\eta_S : S \to a_j S$.

While the operational description in Section 3 details the DoT mechanism, establishing its logical soundness and robustness requires a deeper, formal framework. We leverage Topos Theory (MacLane and Moerdijk, 2012; Johnstone, 2002; Lambek and Scott, 1988), which provides a setting with finite limits, exponentials, and a subobject classifier to interpret intuitionistic logic. This makes it suitable for formalizing the dynamic, evidence-aggregating, and context-dependent reasoning inherent in the DoT process.

An elementary topos $\mathcal{E}$ is a category that encapsulates key properties needed for modeling logical systems and computation:

1. **Finite Limits:** $\mathcal{E}$ has a terminal object 1, binary products $A \times B$, and pullbacks. This allows for combining and constraining information.
2. **Cartesian Closure:** For any objects $A, B \in \mathcal{E}$, there exists an exponential object B^A, representing the internal collection of morphisms $A \to B$. This enables modeling functions, predicates, and higher-order logic.
3. **Subobject Classifier Ω:** There exists an object Ω and a truth arrow $\top : 1 \to \Omega$ such that every monomorphism $m : P \hookrightarrow A$ corresponds uniquely to a characteristic morphism $\chi_m : A \to \Omega$. Ω internalizes the logic of the topos, which is generally a Heyting algebra, supporting intuitionistic reasoning.

Crucially for DoT, presheaf topoi $\mathcal{E} = \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ are complete and cocomplete. For the synthesis step we focus on, the relevant universal construction is a finite limit (conjunction of constraints) in the slice $(\mathcal{E}/S)$; dually, this is a variance-reversed colimit in the information order $\mathrm{Pred}(S) = \mathrm{Sub}(S)^{\mathrm{op}}$. Since slices of a presheaf topos are again topoi, the slice $(\mathcal{E}/S)$ has the finite limits we require (and also all small colimits, should one wish to model disjunctive/quotient-style aggregation).

In what follows, we *fix* a presheaf topos $\mathcal{E} = \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$. This is the category of functors $\mathcal{C}^{\mathrm{op}} \to \mathrm{Set}$ for a small category $\mathcal{C}$. We also fix a designated semantic object $S \in \mathcal{E}$ representing the universe of discourse. Our formalization takes place within the *slice category* $(\mathcal{E}/S)$, where predicate-like propositions are subobjects of S and general typed propositions are objects over S. For concreteness, one may take $\mathcal{C}$ to index problem contexts and $S(c)$ to be the set of admissible semantic states at context c. For the QF-LIA instantiation used in examples, we take $\mathcal{C}$ to be a finite discrete category (a single object in simple cases) and interpret terms componentwise so that interpretation remains decidable.

Definition 4.1 (Categorical Semantics of DoT Components)**.** Within the fixed presheaf topos $\mathcal{E} = \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ and slice $(\mathcal{E}/S)$, fix a Lawvere–Tierney topology $j : \Omega \to \Omega$ with associated universal closure operator $c = (c_A)_{A \in \mathcal{E}}$ on subobjects (extensive, idempotent, monotone, pullback-stable). When discussing only subobjects of S, we write c for the component $c_S : \mathrm{Sub}(S) \to \mathrm{Sub}(S)$.

- **Semantic Space (S):** A base object $S \in \mathcal{E}$ representing the universe of discourse or the space of all possible solutions and intermediate states.
- **Propositions (P):** A proposer node is interpreted as an object $p : P \to S$ of the slice category $(\mathcal{E}/S)$. In the predicate/mono instantiation, p is a monomorphism $P \hookrightarrow S$, and we freely identify it with the corresponding subobject (a "subset" of S).
- **Entailment vs. dependency variance:** In the predicate instantiation, entailment $P \Rightarrow Q$ cor-

responds to inclusion $P \leq Q$ in $\mathrm{Sub}(S)$ (equivalently, the unique morphism $P \to Q$ over S). Operational dependency edges are not, by themselves, semantic entailments. The extracted index category $\mathcal{J}_G$ (Def. 4.3) generates semantic arrows only from certified strengthening/refinement records and certified `@entails` records. Thus a certified strengthening of proposer u by proposer v induces an arrow $v \to u$ in $\mathcal{J}_G$, interpreted as a map $D_G(v) \to D_G(u)$ over S (predicate case: $D_G(v) \leq D_G(u)$), while `@entails src=i dst=k` induces an arrow $i \to k$ interpreted as $D_G(i) \to D_G(k)$. In particular, an operational record `@edge src=u dst=v kind=refine` points from the old node to the new node as provenance, but if it carries a typed strengthening certificate, the induced semantic arrow is $v \to u$. Ordinary `use`, `critique`, and uncertified repair-style `refine` edges record provenance/control flow only.

- **Critiques as judgements (typed):** A `<critic>` node is typed evidence *about* one or more propositions. Semantically, an accepted critique record contributes (i) a validation mark for its target proposer, and optionally (ii) typed arrows, isomorphisms, or path-equality constraints between proposer objects over S (e.g., certified `@entails`/`@eq` records), which become generators or relations in the extracted diagram. In the predicate/mono fragment, certified entailment arrows are inclusions between subobjects; in the general slice fragment, they may be arbitrary morphisms over S. Critic nodes themselves are witnesses for these records rather than objects of the semantic synthesis diagram.
- **Validation as a nucleus/reflection:** On subobjects, validation is modeled by the universal closure $c : \mathrm{Sub}(S) \to \mathrm{Sub}(S)$ induced by j; in particular c is extensive, monotone, idempotent, pullback-stable, and satisfies $c(P \wedge Q) = c(P) \wedge c(Q)$ for finite meets (i.e., c is a nucleus). In the predicate/mono fragment, a proposition is semantically validated iff $c(P) = P$ (i.e., it is c-closed). For a general slice object $X \to S$, validation is represented by applying the induced left-exact sheafification functor $a_j^{/S}$; we do not call arbitrary objects c-closed unless they are subobjects.
- **Strengthening vs. repair:** A strengthening step produces a new proposer object $p' : P' \to S$ equipped with a certified morphism $P' \to P$ over S (interpreted as "P' entails/strengthens P"). In the predicate/mono fragment, this is precisely an inclusion $P' \hookrightarrow P$ in $\mathrm{Sub}(S)$. A repair step after an invalidation need not strengthen the rejected proposition; it contributes to the semantic diagram only through separately certified entailment or equivalence records.
- **Selected Validated Diagram:** Only validated proposer nodes selected by the summarizer enter the synthesis computation; critique nodes provide certified morphisms and equivalence constraints between proposer nodes.

Assumption 4.2 (Fixed, pullback-stable validation modality)**.** There exists a Lawvere–Tierney topology $j : \Omega \to \Omega$ on $\mathcal{E}$ whose induced universal closure operator $c = (c_A)_{A \in \mathcal{E}}$ models validation. Each component $c_A : \mathrm{Sub}(A) \to \mathrm{Sub}(A)$ is extensive, monotone, idempotent, and pullback-stable. All results in this section are relative to this fixed j. Moreover, since c is induced by a Lawvere–Tierney topology, each c_A is a nucleus on $\mathrm{Sub}(A)$: it preserves finite meets. When we apply c to propositions over S, we mean the component c_S. We work throughout in the presheaf topos setting, so the finite limits we use in synthesis exist in $(\mathcal{E}/S)$; the induced functor $a_j^{/S}$ is left exact and therefore preserves these finite limits.

4.1 Semantic Target and Normative Conditions

The following assumptions connect the practical LLM behavior with our formal model. They are idealized, normative conditions that define the target semantics for a well-behaved DoT agent. The

operational mechanisms are designed to make LLM traces more likely to satisfy these conditions upon interpretation.

(A1) Typed proposer nodes admit interpretations as objects over S (as subobjects in the predicate/-mono fragment); certified critique content interprets as arrows/predicates that constrain these objects. Certified strengthening/refinement and `@entails` records correspond to morphisms over S.[1]

(A2) In the predicate/mono fragment, critique-driven validation corresponds to the Lawvere–Tierney closure c on $\mathrm{Sub}(S)$; validated predicate nodes are exactly the c-closed subobjects. In the general-arrow fragment, validated synthesis is computed after applying the induced slice sheafification functor.

(A3) Equivalences and path equalities identified by validated critiques are respected. Formally, we quotient the generated index category by the smallest congruence induced by certified equality records (see Def. 4.3); in the predicate/posetal syntax, `@eq src=i dst=k` abbreviates mutual entailment/equality of proposer subobjects, while in the general slice syntax it requires a certified isomorphism over S.

(A4) For our main result, we consider the fragment in which every selected validated proposer is interpreted as a monomorphism $P \hookrightarrow S$ and every selected validated arrow between proposers is (hence) an inclusion in $\mathrm{Sub}(S)$, so the selected diagram lands in the posetal category $\mathrm{Sub}(S) \subseteq (\mathcal{E}/S)$.

4.2 The Reasoning Diagram and its Synthesis via Finite Limit

The DoT-DAG $G = (V, E)$ specifies the operational trace. The categorical diagram used for synthesis is extracted from the certified semantic records inside that trace.

Definition 4.3 (DoT Index Category $\mathcal{J}_G$)**.** Given a well-formed typed trace with DoT DAG $G = (V, E)$, let $V_{\mathrm{prop}} \subseteq V$ be the subset of proposer nodes. We distinguish the operational DAG from the semantic diagram: ordinary dependency edges do not automatically generate semantic morphisms.

Let $\mathsf{A}_{\mathrm{sem}}(G)$ be the directed multigraph on V_{prop} with the following certified semantic generators:

- an arrow $v \to u$ whenever the trace contains an accepted typed certificate that proposer v strengthens proposer u; for a certified operational record `@edge src=u dst=v kind=refine`, the semantic arrow is therefore opposite to the operational dependency edge;
- an arrow $i \to k$ for each accepted record `@entails src=i dst=k` between proposer nodes;
- in the general slice fragment, inverse arrows for each accepted `@eq` record certified as an isomorphism over S.

Pure `use`, `critique`, and uncertified repair edges are excluded from $\mathsf{A}_{\mathrm{sem}}(G)$.

The semantic index category is

$$\mathcal{J}_G := \mathsf{Free}(\mathsf{A}_{\mathrm{sem}}(G))/\equiv,$$

where $\equiv$ is the smallest arrow congruence generated by certified path-equality records and, for certified isomorphism records, by the inverse equations. In the predicate/posetal fragment, `@eq src=i dst=k` abbreviates mutual entailment and hence equality of subobjects; equivalently, it

[1] This is a strong assumption; our framework is normative: it specifies the target semantics an ideal DoT agent should realize, enabled by the typed serialization in Section 3.2.

identifies the corresponding objects in the thin category. Richer path-equality syntax can be added by extending the same congruence.

For all validated content, $\mathcal{J}_{\text{valid}}$ denotes the finite presentation generated by validated proposer objects and certified semantic arrows whose source and target are validated. Paths through invalidated proposer nodes are not used unless the trace separately certifies an arrow between validated endpoints.

For a particular summarizer node s, let

$$V_\Sigma(s) = \{\, v \in V_{\text{prop}} : \ v \text{ is validated and } \texttt{@edge src=}v\ \texttt{dst=}s\ \texttt{kind=use} \text{ is accepted} \,\}.$$

The synthesis presentation $\mathcal{J}_\Sigma(s)$ is the subpresentation generated by $V_\Sigma(s)$ and the certified semantic arrows and relations whose endpoints lie in $V_\Sigma(s)$. If no summarizer node is specified and one wants a global full-run summary, we set $V_\Sigma = V_{\text{valid}}$ and $\mathcal{J}_\Sigma = \mathcal{J}_{\text{valid}}$ by convention.

Theorem 4.4 (DoT Process as Diagram Construction)**.** A DoT reasoning process generating a well-formed typed DAG $G = (V, E)$, together with a fixed semantic interpretation of typed propositions and certified arrows, defines a functor (a diagram) $D_G : \mathcal{J}_G \to (\mathcal{E}/S)$ with:

- Each typed proposer node v mapped to an object $D_G(v) \to S$ in the slice (a subobject $D_G(v) \hookrightarrow S$ in the predicate/mono fragment);
- Accepted critic records supply certified arrows, isomorphisms, or path equalities between proposer objects; critic nodes are witnesses for these records rather than objects of D_G;
- Each arrow $v \to u$ in $\mathcal{J}_G$ is mapped to a certified morphism over S, witnessing entailment or strengthening coherence (posetal case: an inclusion $D_G(v) \hookrightarrow D_G(u)$).

Functoriality ensures that composite certified dependencies (via paths modulo $\equiv$) are respected in the slice.

Proof Sketch. By Assumption (A1), each typed proposer node v admits an interpretation as an object $D_G(v) \to S$ of the slice $(\mathcal{E}/S)$, and as a subobject in the predicate/mono fragment. The generators of $\mathcal{J}_G$ are certified strengthening/refinement arrows, certified `@entails` arrows, and certified isomorphism/path-equality records. The validator requires each such generator to have a typed semantic witness, so Assumption (A1) assigns it a morphism over S. We define D_G on composite arrows by composition in $(\mathcal{E}/S)$. Certified equality records generate the congruence used in $\mathcal{J}_G$, and Assumption (A3) ensures that equivalent syntactic paths induce equal semantic composites. Hence D_G is well-defined and preserves identities and composition. □

Synthesis by information-colimit (slice-limit) and reflection. The `<summarizer>` aggregates selected validated content. We distinguish an inclusion/posetal setting (our main focus) and a general-arrow setting. In the latter, sheafification induces a base-change: $a_j^{/S} : (\mathcal{E}/S) \to (\mathrm{Sh}_j(\mathcal{E})/a_j S)$, and summaries are read over $a_j S$ via the unit $S \to a_j S$.

Theorem 4.5 (Summarization as finite meet-plus-closure in the reflective subposet)**.** Assume every proposer selected for synthesis is interpreted as a c-closed subobject of S and every certified semantic arrow between selected proposers is an inclusion, so the selected diagram $D_\Sigma = D_G|_{\mathcal{J}_\Sigma}$ lands in the thin category $\mathrm{Sub}(S)$. Let V_Σ be the finite set of validated proposer nodes selected by the summarizer, with the convention that the empty meet is S. The finite-limit summary is

$$\text{Summary}_\Sigma \ = \ \lim_{\mathrm{Sub}(S)} D_\Sigma \ = \ \bigwedge_{v \in V_\Sigma} D_G(v).$$

Equivalently, this is the colimit of the variance-reversed diagram $D_\Sigma^{\mathrm{op}} : \mathcal{J}_\Sigma^{\mathrm{op}} \to \mathrm{Pred}(S) = \mathrm{Sub}(S)^{\mathrm{op}}$. If one writes the reflected form

$$c\Big(\bigwedge_{v \in V_\Sigma} D_G(v)\Big),$$

then the outer c is redundant because c is a nucleus and all inputs are already c-closed.

Proof Sketch. In the predicate fragment, selected validated proposer nodes are subobjects of S. Since $\mathrm{Sub}(S)$ is thin, the limit of a finite diagram is the greatest lower bound of its objects, namely their meet. Equivalently, this meet is the finite join of the corresponding opposite diagram in the information order $\mathrm{Pred}(S) = \mathrm{Sub}(S)^{\mathrm{op}}$. Since each selected proposition is c-closed and c preserves finite meets, the meet is again c-closed; writing the result as $c(\bigwedge_v D_G(v))$ emphasizes compatibility with the reflected construction. □

Corollary 4.6 (General case via slice sheafification)**.** For a general selected diagram $D_\Sigma : \mathcal{J}_\Sigma \to (\mathcal{E}/S)$ (not necessarily posetal), presented by a finite generating graph and a finite set of certified coherence/path-equality relations already satisfied by D_Σ, the synthesized summary in the validated/sheaf slice is

$$\mathrm{Summary}_\Sigma \;\cong\; \lim_{\mathrm{Sh}_j(\mathcal{E})/a_jS} \left(a_j^{/S} \circ D_\Sigma\right) \;\cong\; a_j^{/S}\big(\lim_{\mathcal{E}/S} D_\Sigma\big),$$

where $a_j^{/S} : (\mathcal{E}/S) \to (\mathrm{Sh}_j(\mathcal{E})/a_jS)$ is the left-exact functor induced by sheafification, sending $X \to S$ to $a_jX \to a_jS$. If one wishes to compare the resulting object back to $(\mathcal{E}/S)$, one pulls back along the unit $\eta_S : S \to a_jS$. For a subobject $P \hookrightarrow S$, this inverse image represents the j-closure $c_S(P) \hookrightarrow S$; hence, in the posetal fragment, the pulled-back sheafified limit reduces to the reflected form in Theorem 4.5.

Proof Sketch. Since runs are finite, the extracted semantic shape has a finite graph presentation and finitely many certified path-equality relations. The accepted relations are part of the well-definedness of D_Σ; once they are satisfied, the limit can be computed from finite products and equalizers imposing cone compatibility for the finitely many generating arrows. The slice sheafification functor $a_j^{/S}$ is left exact (as a_j is), hence preserves finite limits. In the posetal fragment, limits in $\mathrm{Sub}(S)$ are meets (intersections). The sheafified mono lives over a_jS; pulling it back along $\eta_S : S \to a_jS$ gives the j-closure of the original meet. Thus the subobject seen back over S is $c_S(\bigwedge_v D_G(v))$, recovering Theorem 4.5. □

4.3 Formal Guarantees: Consistency and Robustness

Theorem 4.7 (Conditional consistency via closure validation)**.** Fix $\mathcal{E} = \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ and the Lawvere–Tierney closure c on $\mathrm{Sub}(S)$. For a finite DoT run and a chosen summarizer, let V_Σ be the finite set of validated, c-closed subobjects selected for synthesis. The following two consistency readings are valid:

1. **Fixed-stage finite family:** If the interpreted family is jointly satisfiable relative to a fixed stage $a \in \mathcal{C}$, i.e. there exists $x \in S(a)$ lying in every $D_G(v)(a)$, then

$$c\left(\bigwedge_{v \in V_\Sigma} D_G(v)\right)(a) \neq \varnothing$$

and the summary is non-initial. Componentwise non-emptiness of the individual $D_G(v)(a)$ is not sufficient; the premise requires a common element.

2. **Model-compact setting:** Let Γ be the set of first-order formulas represented by an idealized validated typed family, and suppose $\mathbb{T}$ is a first-order background theory. If every finite subset of Γ is satisfiable together with $\mathbb{T}$, then compactness gives a model $\mathfrak{M} \models \mathbb{T}$ and an assignment satisfying all formulas in Γ. Thus the conjunction represented by the family is satisfiable in that model. If the chosen semantic interpretation of S and c is evaluated in the same model, extensivity of c preserves satisfiability of the corresponding closed summary. This is a model-theoretic satisfiability statement, not a claim of satisfiability in the intended/standard model, and not a claim of componentwise non-emptiness in a fixed presheaf stage.

Proof Sketch. Case (1): at the fixed stage a, joint satisfiability gives a common element of all interpreted subobjects, so the componentwise finite intersection is non-empty; extensivity of c preserves non-emptiness. Case (2) is the standard compactness theorem for first-order logic, followed by the observation that an extensive closure cannot destroy a realizing assignment when the closure is interpreted in the same semantic structure. □

Corollary 4.8 (Soundness w.r.t. satisfiability)**.** If $[\![\cdot]\!]$ maps into $\mathrm{Sub}(S)$ and the selected validated family $\{P_v\}_{v \in V_\Sigma}$ is jointly satisfiable in the same fixed stage or semantic structure in which the summary is interpreted, then the meet $\bigwedge_v P_v$ is satisfiable. Consequently, by extensivity, the reflected summary $c\,(\bigwedge_v P_v)$ is satisfiable in that same interpretation.

Remark 4.9 (Internal Logic, Modality, and Variance)**.** The internal logic of $\mathcal{E}$ equips $\mathrm{Sub}(S)$ with a Heyting algebra. Validation via a Lawvere–Tierney topology promotes propositions to c-closed ones. In inclusion order, synthesis is a finite meet of constraints; after reversing variance, the corresponding diagram in the opposite information order $\mathrm{Pred}(S) = \mathrm{Sub}(S)^{\mathrm{op}}$ has this same object as a finite colimit. Thus "gluing validated parts" should be read as a finite slice-limit, or equivalently as an information-order colimit in the predicate fragment, not as an ordinary colimit in $\mathrm{Sub}(S)$.

Proposition 4.10 (Robustness under Diagram Isomorphisms)**.** Let $D_{\Sigma,1} : \mathcal{J}_{\Sigma,1} \to (\mathcal{E}/S)$ and $D_{\Sigma,2} : \mathcal{J}_{\Sigma,2} \to (\mathcal{E}/S)$ represent selected validated reasoning steps from two different runs. If there is an isomorphism of diagrams, then their slice-limits (equivalently, information-colimits in the predicate fragment) are isomorphic:

$$\lim D_{\Sigma,1} \cong \lim D_{\Sigma,2}.$$

This implies that the synthesized semantic content depends on the abstract structure of the selected validated reasoning diagram, not on incidental variations that preserve this structure.

Proof Sketch. This is a direct consequence of the universal property of a limit. If two diagrams are isomorphic, there is a canonical isomorphism between their respective limits. □

4.4 Immediate Consequences

The formalization of the DoT synthesis step as a colimit in the variance-reversed information order (equivalently, as meet-plus-closure under inclusion) entails several immediate and desirable properties, grounded in the lattice-theoretic structure of subobjects and the properties of the closure operator c.

Proposition 4.11 (Properties of Synthesis)**.** Let $\mathcal{P} = \{P_v\}_{v \in V_\Sigma}$ be a finite selected family of predicate-fragment subobjects. Define the reflected synthesis operation by $\mathrm{Summary}(\mathcal{P}) = c(\bigwedge_{P \in \mathcal{P}} P)$; when all selected inputs are validated, this equals the ordinary meet because they are c-closed and c is a nucleus. The operation exhibits the following properties:

1. **Finiteness in practice:** as runs are finite, all meets are finite; infinitary variants require compactness assumptions and are outside our core claims.
2. **Monotonicity (information order):** Adding a new selected proposition P_{new} can only refine (never weaken) the reflected summary. In inclusion order,
$$\mathrm{Summary}(\mathcal{P} \cup \{P_{\text{new}}\}) \subseteq \mathrm{Summary}(\mathcal{P}).$$
3. **Idempotence (finite meets):** The system is robust to redundant validation. Re-processing already validated information over finite families does not alter the conclusion:
$$\mathrm{Summary}(\mathcal{P}) = c\left(\bigwedge_{P \in \mathcal{P}} c(P)\right) = c\left(\bigwedge_{P \in \mathcal{P}} P\right).$$
The first equality uses finite-meet preservation of c; when validated propositions are already c-closed ($P = c(P)$), the statement is immediate.
4. **Conservativity (Redundancy Elimination):** If a validated proposition P_w is already *no stronger than* the others' conjunction (i.e., $P_w \supseteq \bigwedge_{P \in \mathcal{P} \setminus \{P_w\}} P$), its explicit inclusion does not change the summary.
$$\mathrm{Summary}(\mathcal{P}) = \mathrm{Summary}(\mathcal{P} \setminus \{P_w\}).$$

Proof. These properties are direct consequences of the underlying mathematics. Monotonicity follows from the monotonicity of $\bigwedge$ and c. Idempotence follows from finite-meet preservation of c, and is immediate for already c-closed inputs. Conservativity follows because if P_w contains the meet of the others, it does not change that meet. □

Proposition 4.12 (Greatest c-closed Lower Bound (Canonicity))**.** Let $\mathcal{P} = \{P_i\}_{i=1}^n \subseteq \mathrm{Sub}(S)$ be a finite family of selected validated subobjects (so $c(P_i) = P_i$ for all i), and assume c is a nucleus (finite-meet preserving; Assumption 4.2). Then the meet $\bigwedge_{i=1}^n P_i$ is itself c-closed and is the *greatest c-closed lower bound* of $\mathcal{P}$ (in inclusion order). In particular,
$$c\Big(\bigwedge_{i=1}^{n} P_i\Big) = \bigwedge_{i=1}^{n} P_i.$$

Proof. Since c preserves finite meets and fixes each P_i, we have
$$c(\bigwedge_i P_i) = \bigwedge_i c(P_i) = \bigwedge_i P_i,$$
so the meet is c-closed. If X is c-closed and $X \leq P_i$ for all i, then $X \leq \bigwedge_i P_i$ by the universal property of the meet. □

Proposition 4.13 (Generalization of Linear Reasoning)**.** A linear Chain-of-Thought (CoT) process corresponds to a special case of a DoT diagram. Suppose the selected, operationally ordered validated propositions are $P_1, \ldots, P_n$, and each later step strengthens the previous one, so that
$$P_1 \supseteq P_2 \supseteq \cdots \supseteq P_n \quad \text{in } \mathrm{Sub}(S).$$

Equivalently, the semantic inclusion arrows point from the later stronger proposition to the earlier weaker one:

$$P_n \longrightarrow P_{n-1} \longrightarrow \cdots \longrightarrow P_1.$$

Then the synthesis simplifies to the final step:

$$\text{Summary}(\{P_1, \ldots, P_n\}) = P_n.$$

Proof. For a chain $P_1 \geq \cdots \geq P_n$ under inclusion, their meet is $\bigwedge_{i=1}^{n} P_i = P_n$. Applying c gives Summary $= c(P_n)$. Since P_n is validated, it is c-closed, so $c(P_n) = P_n$. □

Proposition 4.14 (Composition of Branches)**.** For any two finite selected families A and B, the overall summary of their union is the validated conjunction of their individual summaries (equivalently, the join in the information order). This identity is lattice-theoretic and does not require probabilistic or causal independence.

$$\text{Summary}(A \cup B) = c(\text{Summary}(A) \wedge \text{Summary}(B)) = \text{Summary}(A) \wedge \text{Summary}(B).$$

Proof. By definition, Summary$(A \cup B) = c((\bigwedge_{v \in A} P_v) \wedge (\bigwedge_{w \in B} P_w))$. Since c is a nucleus (finite-meet preserving), $c(X \wedge Y) = c(X) \wedge c(Y)$, and thus

$$\text{Summary}(A \cup B) \;=\; c\left(\bigwedge_{v \in A} P_v\right) \;\wedge\; c\left(\bigwedge_{w \in B} P_w\right) \;=\; \text{Summary}(A) \wedge \text{Summary}(B),$$

which is equivalent to the displayed formula because both individual summaries are c-closed. □

4.5 Bridging Formalism and LLM Generation

It is crucial to understand the relationship between this formal topos-theoretic model and the actual behavior of an LLM. The LLM does not explicitly perform computations within a topos. Instead:

- The topos framework provides the normative semantic model. It defines what constitutes sound, consistent, and robust synthesis. Theorems 4.5, 4.7, and Proposition 4.10 describe desirable properties of an ideal reasoning process.
- The LLM, trained on DoT-structured data (using the serialization from Section 3.2), learns to generate text sequences that functionally approximate the operations described by the formalism. The generated sequence induces an abstract diagram that is then interpreted under Assumptions (A1)–(A4).
- Specifically, the `<summarizer>` role learns to generate text that effectively acts like the finite-limit/information-colimit described above: it synthesizes information from selected validated precursor nodes, respects their certified dependencies, and aims for a coherent, non-redundant aggregation.
- The fidelity of this approximation depends heavily on the training data and model capacity. The topos model provides a precise target against which the LLM's reasoning behavior can be evaluated. One could design specific training objectives, such as a discriminative loss that penalizes generated summaries whose typed content violates the entailments dictated by the finite-limit construction.

This formalism offers a rigorous language for defining correctness criteria and provides a theoretical target for DoT behavior. Operational invalidation can either be modeled (i) as the absence of a validated inclusion (posetal fragment), or (ii) via a separate counterevidence object $I \in \mathrm{Sub}(S)$ with summaries computed as $c((\bigwedge \text{valid}) \wedge \neg I)$ in the Heyting algebra $\mathrm{Sub}(S)$. We adopt the posetal refinement fragment for the main development and leave full revision semantics to future work.

4.6 Separation from Linear Chain-of-Thought

Theorem 4.15 (Structural separation from linear CoT)**.** Let $\mathcal{E} = \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$, fix $S \in \mathcal{E}$, and assume validated arrows are inclusions in $\mathrm{Sub}(S)$ (posetal case). Suppose a DoT summarizer selects two validated, incomparable propositions $P, Q \in \mathrm{Sub}(S)$ (i.e., $P \not\leq Q$ and $Q \not\leq P$). Consider any attempt to faithfully embed this selected validated diagram into a linear Chain-of-Thought (a single chain of inclusions) via an order-preserving and order-reflecting functor that maps P and Q to distinct selected chain objects. No such embedding exists. Consequently, the two-branch DoT diagram cannot be represented as a faithful chain without changing the diagram; a linear trace may compute an equivalent conjunction only by adding or replacing nodes, not by embedding the original branching structure.

Proof sketch. Interpret the selected validated proposer subdiagram as a poset-category. A linear CoT is a chain (total order) in $\mathrm{Sub}(S)$. To rule out degenerate collapse maps, we require an order embedding, i.e., a functor that is order-preserving and order-reflecting (equivalently, full and faithful for posets, and injective on the selected objects). In a chain, any two distinct images are comparable; by order-reflection this would force P and Q to be comparable in $\mathrm{Sub}(S)$, contradicting incomparability. Hence no order embedding of the selected validated DoT poset into a chain exists. The DoT summary is $c(P \wedge Q)$ by Thm. 4.5; a chain can contain a later node denoting this meet, but doing so adds/replaces semantic content rather than faithfully embedding the original two-branch diagram. □

5 Conclusion

This paper introduced the Diagram of Thought (DoT), a framework that internalizes complex reasoning as DAG construction within a single auto-regressive LLM, guided by role-specific tokens and enforced by a lightweight, controller-light validator. We showed how DoT unifies proposition generation, critique, repair, and summarization without a heavyweight search/planning controller. We established a normative formalization using Topos Theory, where the synthesis of selected validated evidence corresponds to computing a finite limit in the slice (equivalently, a variance-reversed information-order colimit in the predicate fragment) and reflecting it along a Lawvere–Tierney topology when necessary.

Crucially, we moved beyond informal assumptions by specifying a typed serialization with online validation, decidable checks for selected typed fragments, a monotone state-update discipline, and support for multi-premise critiques. We separated operational critique records from the Lawvere–Tierney modality they are interpreted against. Our correctness claims are conditional on these checkable and auditable mechanisms, providing a solid bridge between the operational system and its semantic model.

This topos-theoretic perspective provides several key benefits:

- It assigns clear mathematical meaning (subobjects and slice diagrams in $(\mathcal{E}/S)$) to DoT components.
- It formalizes synthesis via closure-based validation together with a variance-reversed information-colimit / slice-limit construction, with a precise split between the posetal and general-arrow settings (Theorem 4.5, Cor. 4.6).
- It demonstrates semantic invariance under isomorphic rearrangements (Proposition 4.10) and compositional gluing via pullbacks/fiber products of limits (Proposition C.3).
- It structurally extends linear CoT in the posetal setting (Proposition 4.13 and Theorem 4.15), matching intuitive gains from branching with a crisp categorical witness.

In summary, the primary advantages of DoT include an auditable reasoning trace, explicit compositional structure, and a clear theoretical target.

References

Maciej Besta, Nils Blach, Ales Kubicek, Robert Gerstenberger, Michal Podstawski, Lukas Gianinazzi, Joanna Gajda, Tomasz Lehmann, Hubert Niewiadomski, Piotr Nyczyk, et al. Graph of thoughts: Solving elaborate problems with large language models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 38, pages 17682–17690, 2024.

Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

Jie Huang and Kevin Chen-Chuan Chang. Towards reasoning in large language models: A survey. *arXiv preprint arXiv:2212.10403*, 2022.

Peter T Johnstone. *Sketches of an Elephant: A Topos Theory Compendium: Volume 2*, volume 2. Oxford University Press, 2002.

Takeshi Kojima, Shixiang Shane Gu, Machel Reid, Yutaka Matsuo, and Yusuke Iwasawa. Large language models are zero-shot reasoners. *Advances in neural information processing systems*, 35: 22199–22213, 2022.

Joachim Lambek and Philip J Scott. *Introduction to higher-order categorical logic*, volume 7. Cambridge University Press, 1988.

Saunders MacLane and Ieke Moerdijk. *Sheaves in geometry and logic: A first introduction to topos theory*. Springer Science & Business Media, 2012.

Aman Madaan, Niket Tandon, Prakhar Gupta, Skyler Hallinan, Luyu Gao, Sarah Wiegreffe, Uri Alon, Nouha Dziri, Shrimai Prabhumoye, Yiming Yang, et al. Self-refine: Iterative refinement with self-feedback. *arXiv preprint arXiv:2303.17651*, 2023.

Hugo Touvron, Thibaut Lavril, Gautier Izacard, Xavier Martinet, Marie-Anne Lachaux, Timothée Lacroix, Baptiste Rozière, Naman Goyal, Eric Hambro, Faisal Azhar, et al. Llama: Open and efficient foundation language models. *arXiv preprint arXiv:2302.13971*, 2023.

Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. *arXiv preprint arXiv:2203.11171*, 2022.

Jason Wei, Xuezhi Wang, Dale Schuurmans, Maarten Bosma, Fei Xia, Ed Chi, Quoc V Le, Denny Zhou, et al. Chain-of-thought prompting elicits reasoning in large language models. *Advances in neural information processing systems*, 35:24824–24837, 2022.

Shunyu Yao, Dian Yu, Jeffrey Zhao, Izhak Shafran, Thomas L Griffiths, Yuan Cao, and Karthik Narasimhan. Tree of thoughts: Deliberate problem solving with large language models. *arXiv preprint arXiv:2305.10601*, 2023.

Yang Yuan. On the power of foundation models. In *International Conference on Machine Learning*, pages 40519–40530. PMLR, 2023.

Yifan Zhang, Jingqin Yang, Yang Yuan, and Andrew Chi-Chih Yao. Cumulative reasoning with large language models. *arXiv preprint arXiv:2308.04371*, 2023.

Appendix

A Worked Example Trace

This example shows DoT applied to a problem with multiple independent conditions, demonstrating branching and invalidation.

```
@node id=1 role=problem
Is the point (4, 6) inside the region defined by x > 2, y > 5, and x < 10?
@node id=2 role=proposer
@edge src=1 dst=2 kind=use
@prop id=2 { "gt":  [4, 2] }
First, I will check the condition x > 2.  For the point (4, 6), x=4.  The inequality
is 4 > 2, which is true.
@node id=3 role=critic
@edge src=2 dst=3 kind=critique
The arithmetic is correct.  This condition holds.
@status target=2 mark=validated
@node id=4 role=proposer
@edge src=1 dst=4 kind=use
@prop id=4 { "gt":  [6, 5] }
Next, I will check the condition y > 5.  For the point (4, 6), y=6.  The inequality
is 6 > 5, which is also true.
@node id=5 role=critic
@edge src=4 dst=5 kind=critique
Correct.  The second condition holds.
@status target=4 mark=validated
@node id=6 role=proposer
@edge src=1 dst=6 kind=use
@prop id=6 { "gt":  [6, 0] }
Let me try an alternative, irrelevant check.  Maybe the condition was y > 0.  For y=6,
6 > 0 is true.
@node id=7 role=critic
@edge src=6 dst=7 kind=critique
This proposition is arithmetically correct, but it addresses a condition not present
in the problem statement.  This path is irrelevant.
@status target=6 mark=invalidated
@node id=8 role=proposer
@edge src=1 dst=8 kind=use
@prop id=8 { "lt":  [4, 10] }
Finally, I check the third condition, x < 10.  For x=4, 4 < 10 is true.
@node id=9 role=critic
@edge src=8 dst=9 kind=critique
This is correct.  The final condition is met.
@status target=8 mark=validated
@node id=10 role=summarizer
@edge src=2 dst=10 kind=use
@edge src=4 dst=10 kind=use
```

```
@edge src=8 dst=10 kind=use
Summary:  All three conditions (x > 2, y > 5, and x < 10) are met for the point (4,
6).  Therefore, the point is inside the specified region.  The validated propositions
are ID=2, ID=4, and ID=8.
```

B DoT Serialization and Validation Details

This section provides the complete details for the typed serialization, validation, and extraction mechanisms summarized in Section 3.2.

Serializer. We present a node-first, into-the-current-node serializer that guarantees acyclicity of the operational provenance DAG and supports online checks. The serializer is optional for usability, but recommended whenever formal guarantees or auditability are invoked. When it is disabled, DoT traces remain interpretable as natural language; the formal guarantees in Section 4 apply to the typed subtrace (if any), and free text is semantically inert.

B.1 Grammar and Record Specification

Node identifiers and roles. Each node carries a fresh natural-number ID and an explicit role:

$$\texttt{@node id=}\langle n\rangle\ \texttt{role=\{problem,proposer,critic,summarizer\}}.$$

IDs are strictly increasing in emission order.

Typed edges and emission order. Edges are explicit, typed dependencies into the current node j:

$$\texttt{@edge src=}\langle i\rangle\ \texttt{dst=}j\ \texttt{kind=\{refine,critique,use\}},\quad i < j.$$

Well-formedness requires that sources `src` refer only to previously emitted node IDs. For critiques, the dependency direction is proposer $\rightarrow$ critic (the critic depends on the proposition it evaluates).

Admissible role/kind pairs (type table). Let $\mathrm{r}(\cdot)$ be node roles. Allowed triples for an `@edge src=i dst=j kind=k` are:

- `kind=critique`: $\mathrm{r}(i) = \texttt{proposer}$, $\mathrm{r}(j) = \texttt{critic}$. A critic node's block may only contain edges of this kind.
- `kind=refine`: $\mathrm{r}(i) \in \{\texttt{proposer}, \texttt{critic}\}$, $\mathrm{r}(j) = \texttt{proposer}$. This edge is procedural unless the validator also accepts a typed strengthening witness; repair after invalidation is not assumed to entail the rejected proposition. If both endpoints are proposers and the witness certifies that the new proposer j strengthens the old proposer i, extraction adds the semantic arrow $j \rightarrow i$, opposite to the operational edge direction.
- `kind=use`: $\mathrm{r}(i) \in \{\texttt{problem}, \texttt{proposer}\}$, $\mathrm{r}(j) \in \{\texttt{proposer}, \texttt{summarizer}\}$. A `problem` source is contextual and is not itself included in the validated-proposition meet. Critics are relational and cannot be a source for `use` edges; instead, their semantic effect is reified via certified `@entails` or `@eq` records.

Arrow declarations (entailments) between proposers. Critiques can declare typed entailments between proposer nodes that become arrows in the extracted diagram:

$$\texttt{@entails src=}\langle i\rangle\ \texttt{dst=}\langle k\rangle\ [\texttt{witness=}\langle j\rangle],$$

with typing requirements: $\mathrm{r}(i) = \mathrm{r}(k) = \texttt{proposer}$, and the `witness`, if written, must be a `critic` node j whose typed certificate is accepted by the validator. If the witness field is omitted, the current node must be a critic and is used as the witness. The intended meaning is $P_i \Rightarrow P_k$, hence a morphism $D_G(i) \to D_G(k)$ over S (an inclusion $P_i \leq P_k$ in the predicate fragment). Entailments are accepted only when certified by the typed witness and, in solver-backed fragments, by the corresponding entailment check.

Equivalence declarations. The basic syntax

$$\texttt{@eq src=}\langle i\rangle\ \texttt{dst=}\langle k\rangle\ [\texttt{witness=}\langle j\rangle]$$

is an abbreviation for mutual entailment in the predicate/posetal fragment. In the general slice fragment, it is accepted only when the witness certifies an isomorphism over S or explicit inverse/path-equality data. It is not, by itself, a general path-equality language. If one implements richer path equalities, those records are added to the same congruence used in Def. 4.3.

Validation marks. Critiques emit a status for their target proposition:

$$\texttt{@status target=}\langle k\rangle\ \texttt{mark=\{validated,invalidated\}}\ [\texttt{just=}\langle i\rangle].$$

Only propositions with a `validated` status can contribute objects to the categorical synthesis diagram, and only when selected by the summarizer. A status record is well formed only when emitted by a critic that has a `kind=critique` edge from the target, and the optional `just` field must name that critic. We adopt a monotone state discipline: a proposition's status may be set only once (first-writer wins).

Lexical discipline (fencing). Typed records must begin with the reserved sigil `@`. Free-form natural-language text lines must not begin with `@`. This ensures unambiguous lexing. To embed arbitrary text (including leading `@`) we use a length-prefixed fence that is streaming-safe:

$$\texttt{@@len=}\langle N\rangle\texttt{@@}\ \underbrace{\langle\text{exactly } N \text{ bytes of raw text}\rangle}_{\text{may contain @ and newlines}}.$$

Concrete grammar (BNF). Let $\mathbb{N}$ be decimal naturals and Role be the set of roles.

$$
\begin{array}{rcl}
\mathsf{Trace} & ::= & \mathsf{NodeBlock}^{+} \\
\mathsf{NodeBlock} & ::= & \mathsf{Node}\ ;\ \mathsf{BlockItem}^{*} \\
\mathsf{BlockItem} & ::= & \mathsf{Edge} \mid \mathsf{Status} \mid \mathsf{Entails} \mid \mathsf{Eq} \mid \mathsf{PropBlock} \mid \mathsf{Free} \\
\mathsf{Free} & ::= & \mathsf{Text} \mid \mathsf{Esc} \\
\mathsf{Node} & ::= & \texttt{@node id=}\mathbb{N}\ \ \texttt{role=(problem|proposer|critic|summarizer)} \\
\mathsf{Edge} & ::= & \texttt{@edge src=}\mathbb{N}\ \ \texttt{dst=}\mathbb{N}\ \ \texttt{kind=(refine|critique|use)} \\
\mathsf{Status} & ::= & \texttt{@status target=}\mathbb{N}\ \ \texttt{mark=(validated|invalidated)}\ [\ \texttt{just=}\mathbb{N}] \\
\mathsf{Entails} & ::= & \texttt{@entails src=}\mathbb{N}\ \ \texttt{dst=}\mathbb{N}\ [\ \texttt{witness=}\mathbb{N}] \\
\mathsf{Eq} & ::= & \texttt{@eq src=}\mathbb{N}\ \ \texttt{dst=}\mathbb{N}\ [\ \texttt{witness=}\mathbb{N}] \\
\mathsf{PropBlock} & ::= & \texttt{@prop id=}\mathbb{N}\ \mathsf{Prop} \\
\mathsf{Prop} & ::= & \text{solver-checkable typed formula in the selected background theory} \\
\mathsf{Text} & ::= & \text{arbitrary natural-language line not starting with @} \\
\mathsf{Esc} & ::= & \texttt{@@len=}\mathbb{N}\texttt{@@}\ \mathsf{Bytes}^{\{\mathbb{N}\}}
\end{array}
$$

The grammar permits free text before or after typed records inside a node block. The validator, not the BNF alone, enforces role-specific side conditions such as "status records may only be emitted by critic blocks" and "summarizer use-edges may point only to validated proposer nodes, except for optional contextual edges from the problem node."

B.2 Validation and Extraction

Well-formedness judgment. We write $\Gamma \vdash \mathsf{Trace}\ \mathsf{ok}$, where the context $\Gamma = (\mathsf{seen}, \mathsf{role}, \mathsf{state}, \mathsf{current})$ tracks: seen node IDs, each node's role, node states, and the current node being emitted. Selected rules:

$$\frac{n > \max(\mathsf{seen}(\Gamma)) \quad r \in \{\texttt{problem}, \texttt{proposer}, \texttt{critic}, \texttt{summarizer}\}}{\Gamma \vdash \texttt{@node id=}n\ \texttt{role=}r\ \ \mathsf{ok}} \quad (\textsc{Node-Intro})$$

$$\frac{i \in \mathsf{seen}(\Gamma) \quad i < j \quad \texttt{dst} = j = \mathsf{current}(\Gamma) \quad (\mathrm{r}(i), \mathrm{r}(j), k)\ \text{admissible}}{\Gamma \vdash \texttt{@edge src=}i\ \texttt{dst=}j\ \texttt{kind=}k\ \ \mathsf{ok}} \quad (\textsc{Edge-Intro})$$

$$\frac{k \in \mathsf{seen}(\Gamma) \quad \mathsf{role}(k) = \texttt{proposer} \quad \mathsf{state}(k) = \texttt{active} \quad \mathsf{role}(\mathsf{current}(\Gamma)) = \texttt{critic} \quad k \in \mathsf{critiqueTargets}(\mathsf{current}(\Gamma))}{\Gamma \vdash \texttt{@status target=}k\ \texttt{mark=}m\ \ \mathsf{ok}}$$

Online validation. We employ a lightweight online validator V with finite control over record kinds and register-based side conditions: (i) a monotone counter for the next ID, (ii) hash maps for roles and states, (iii) a table of critique targets and accepted typed witnesses, and (iv) a congruence-closure structure for equalities. Given a candidate token, V performs local checks (e.g., $\texttt{src} < \texttt{dst} = \texttt{current\_id}$) and, in typed fragments, calls the selected decision procedure for entailment/equality witnesses. At inference, this validator provides masks to the LLM decoder, ensuring only well-formed sequences are generated.

Extraction map. Given a well-formed trace H, the syntactic extraction map $\Phi_{\mathrm{syn}}(H)$ returns: (i) a finite typed DAG $G = (V, E)$; (ii) the validated proposer subgraph; (iii) for each summarizer node s, the selected synthesis presentation $\mathcal{J}_\Sigma(s)$ generated by validated proposer nodes with accepted `use` edges into s; and (iv) an index category $\mathcal{J}_G$ generated by certified strengthening/refinement arrows, certified `@entails` arrows, and certified equivalence/path-equality relations as in Def. 4.3.

Operational `use`, `critique`, and uncertified repair edges remain in the provenance DAG but do not become semantic arrows. After fixing a semantic interpretation $[\![\cdot]\!]$ for typed proposition blocks and certified arrows, $\Phi_{[\![\cdot]\!]}(H)$ returns the corresponding diagram $D_G : \mathcal{J}_G \to (\mathcal{E}/S)$ and its selected subdiagrams $D_\Sigma(s)$.

Theorem B.1 (Totality and Determinism of Extraction)**.** Any well-formed trace H satisfying the ID and edge-typing rules yields a unique typed DAG G, unique selected synthesis presentations $\mathcal{J}_\Sigma(s)$ for summarizer nodes, and a unique syntactic index category $\mathcal{J}_G$ under Φ_{syn}. After fixing a semantic interpretation $[\![\cdot]\!]$, it yields a unique diagram $D_G : \mathcal{J}_G \to (\mathcal{E}/S)$ and selected subdiagrams $D_\Sigma(s)$. Excluding solver-call time, extraction is linear up to dictionary operations in the absence of nontrivial equality/path-equality declarations; with union-find equivalence classes it runs in $O(|H|\,\alpha(|H|))$ time and $O(|H|)$ space. Validation adds the sum of the selected theory solver costs. When general path equalities are present, we maintain congruence-closure on arrows (e.g., via e-graphs); this is near-linear in typical traces but can be super-linear in the worst case, depending on the signature and saturation strategy.

B.3 Operational Semantics and Meta-Theory

Standing scope. We treat DoT's semantics as a normative target: the LLM approximates the finite-limit/information-colimit behavior, while V ensures the typed fraction is well-formed and auditable.

Proposition B.2 (Relative soundness to typed subtrace)**.** Any conclusion in Section 4 is a function of the extracted typed diagram $\Phi(H)$ alone; untyped/free text does not affect the semantics.

We give selected small-step rules over states (G, σ, H) where G is the partial DAG, σ the node-state map, and H the emitted prefix.

Rules (sketch). PROPOSER-INTRO: on a well-typed `@node` with role=`proposer`, extend G with a vertex v, set $\sigma(v) = \mathsf{active}$. CRITIC-INTRO: on role=`critic`, add the critic node. VALIDATE: on `@status` with `mark=validated` for target u where $\sigma(u) = \mathsf{active}$, set $\sigma(u) = \mathsf{validated}$. INVALIDATE: on `@status` with `mark=invalidated` for target u where $\sigma(u) = \mathsf{active}$, set $\sigma(u) = \mathsf{invalidated}$.

Theorem B.3 (Order-Invariance (semantic; typed-content invariant))**.** Let a well-posed trace be one that (i) obeys the NodeBlock grammar, and (ii) contains at most one `@status` record for each proposer node ID (single-assignment). Normalize typed records by resolving omitted `witness` and `just` fields to their enclosing critic block and by discarding free text. Consider two well-posed traces, H_1 and H_2, that contain the same multiset of normalized typed records (`@node`, `@edge`, `@status`, `@entails`, `@eq`) and induce the same dependency partial order on those records after quotienting by validated equalities. Then their extracted index categories are isomorphic, $\mathcal{J}_{G_1} \cong \mathcal{J}_{G_2}$. For corresponding summarizer nodes, the selected synthesis presentations are also isomorphic. In the posetal case, the resulting subobjects are equal; in the general case, their reflected slice-limits are isomorphic.

B.4 Algorithm and Implementation Details

The state of the reasoning process (the DAG) is implicitly encoded within the auto-regressive history H_t. The LLM conditions its prediction on this history, using its attention mechanism to represent the current state of the DoT graph. Algorithm 1 provides a high-level sketch.

Algorithm 1 Diagram of Thought (DoT) Generation Process

Input: Problem statement $\mathcal{P}$
Initialize generation history H with serialized problem statement for node v_1; set current node $j \leftarrow 1$.
Initialize node states (e.g., in a dictionary) $\sigma[v_1] \leftarrow$ initial.
while termination condition not met (e.g., max length, `<summarizer>` generated) **do**
    Predict next role token $r \in T_{\text{roles}}$ based on history H: $r \sim \text{LM}(H)$.
    Append r to H.
    if $r =$ `<proposer>` **then**
        Emit `@node id=` $j{+}1$ `role=proposer`; set $j \leftarrow j{+}1$.
        Emit zero or more well-typed `@edge src=` i `dst=` j with $i < j$.
        Generate proposition text P_j. Append records and text to H.
        Update state: $\sigma[j] \leftarrow$ active.
    else if $r =$ `<critic>` **then**
        Emit `@node id=` $j{+}1$ `role=critic`; set $j \leftarrow j{+}1$.
        Emit one or more `@edge src=` k_m `dst=` j `kind=critique`.
        Generate critique text C_j; then emit one or more `@status target=` k `mark=validated|invalidated`.
        Optionally emit certified `@entails` or `@eq` records.
        Append records and text to H.
        Update target states monotonically: if $\sigma[k] =$ active, set $\sigma[k] \leftarrow m$.
    else if $r =$ `<summarizer>` **then**
        Emit `@node id=` $j{+}1$ `role=summarizer`; set $j \leftarrow j{+}1$.
        Emit zero or more `@edge src=` i `dst=` j `kind=use` from selected validated proposer nodes (and optionally the problem node as context).
        Generate summary text S. Append records and text to H.
        Set termination condition to true.
    end if
end while
Output: Final text S from the summarizer node v_S.

Controller-light decoding. Decoding uses a deterministic, state-dependent mask derived from the validator's finite control, registers, and local typing maps. Illegal token classes are never sampled. There is no branching search or external resampling loop. The only state external to the LM is the validator's finite map store and any solver state needed for the chosen typed fragment.

Auxiliary supervision for summaries-as-limits. To align the `<summarizer>` with the normative target, one can add an auxiliary loss that penalizes violations of literals that are entailed by the finite meet of selected validated propositions. This complements the standard maximum-likelihood training on full traces.

C Additional Formal Details

C.1 From Critique Schemas to a Nucleus

Critique schemas do not automatically determine a Lawvere–Tierney topology. To avoid this gap, the main text assumes a fixed topology j and its universal closure operator. When one wants to derive such a topology from typed critique schemas, only the modality-producing part of the schema should be compiled into pullback-stable closure sequents; certified entailments, equivalences, and exclusions remain separate typed records unless explicitly encoded as such sequents. Thus the items below are constraints on a topology, not arbitrary closure rules on the single lattice $\mathrm{Sub}(S)$:

- **Certified entailment** (LE): a validated critique may introduce a morphism $P \to Q$ over S (an inclusion $P \leq Q$ in the predicate fragment). This is a semantic arrow in $\mathcal{J}_G$; it is not, by itself, a closure axiom.
- **Modal validation sequent**: if a schema is intended to enlarge the closure of a predicate, it is compiled as a pullback-stable lower-bound sequent $Q \leq c_A(P)$ for suitable subobjects $P, Q \hookrightarrow A$.
- **Equivalence identification** (EQ): a validated critique emitting mutual entailments in the predicate fragment or an isomorphism/equivalence record between proposer objects in the general slice fragment. This is handled by certified arrows and congruence equations, not by closure alone.
- **Type refinement** (TR): narrows a subobject P by pullback along a mono $R \hookrightarrow S$, producing $P \wedge R$ and a certified strengthening arrow $P \wedge R \to P$.

The local operators on a topos form a complete lattice. For the closure constraints used here, the relevant modality sequents are lower-bound closure requirements of the form $Q \leq c_A(P)$, closed under pullback. Satisfaction of such sequents is stable under arbitrary meets of local operators under the usual lattice order; hence the satisfying topologies, when nonempty, have a least element. For purely lower-bound closure constraints the chaotic topology satisfies the constraints, so nonemptiness is automatic. Equality, isomorphism, and exclusion constraints are not generated by this lattice argument alone; they must be certified separately or compiled into stable closure sequents whose consistency has been checked. The associated universal closure operator is then extensive, monotone, idempotent, pullback-stable, and finite-meet preserving by construction. This is the limited sense in which critique schemas can induce a validation modality.

Lemma C.1 (Rule closure & nucleus completion)**.** Let $\mathcal{R}$ be a set of pullback-stable lower-bound closure sequents $Q_r \leq c_{A_r}(P_r)$ generated by the modality-producing part of the typed critique schemas. If at least one Lawvere–Tierney topology satisfies $\mathcal{R}$, and if satisfaction of $\mathcal{R}$ is closed under meets of local operators (as it is for these lower-bound sequents), let $j_{\mathcal{R}}$ be the meet of all satisfying topologies. Then $j_{\mathcal{R}}$ is the least topology satisfying $\mathcal{R}$. Its associated universal closure operator $c^{\mathcal{R}} = (c^{\mathcal{R}}_A)_{A \in \mathcal{E}}$ is extensive, monotone, idempotent, pullback-stable, and finite-meet preserving. In particular, $c^{\mathcal{R}}_S$ is a nucleus on $\mathrm{Sub}(S)$.

C.2 Further Categorical Results

Lemma C.2 (Slice reflection and finite-limit stability)**.** Let $a_j : \mathcal{E} \to \mathrm{Sh}_j(\mathcal{E})$ be sheafification, which is left exact. The induced functor on slices

$$a_j^{/S} : (\mathcal{E}/S) \to (\mathrm{Sh}_j(\mathcal{E})/a_j S), \qquad (X \to S) \mapsto (a_j X \to a_j S),$$

is also left exact, and hence preserves finite limits. Moreover, for a mono $m : P \hookrightarrow S$, the inverse image in $\mathcal{E}$ of the mono $a_j m : a_j P \hookrightarrow a_j S$ along the unit $\eta_S : S \to a_j S$ represents the j-closure

$c_S(P) \hookrightarrow S$. This is the stability property required for the synthesis construction in Cor. 4.6 (finite limits in $(\mathcal{E}/S)$ followed by $a_j^{/S}$, and comparison back to $(\mathcal{E}/S)$ by pullback along η_S).

Proposition C.3 (Gluing validated diagrams via pullbacks of limits)**.** Let $D_1 : \mathcal{J}_1 \to (\mathcal{E}/S)$ and $D_2 : \mathcal{J}_2 \to (\mathcal{E}/S)$ be selected validated diagrams whose overlap is a common subdiagram $K : \mathcal{J}_K \to (\mathcal{E}/S)$. Then, in a presheaf topos, there is a canonical isomorphism

$$\lim(D_1 \cup_K D_2) \;\cong\; \lim(D_1) \;\times_{\lim(K)}\; \lim(D_2),$$

whenever the displayed union is the pushout of finite index shapes along the common subshape and the two diagrams agree on K. Hence, after reflection, the overall summary satisfies

$$\mathrm{Summary}(D_1 \cup_K D_2) \;\cong\; a_j^{/S}\big(\lim(D_1) \;\times_{\lim(K)}\; \lim(D_2)\big).$$

Proof sketch. Giving a cone over the union diagram is equivalent to giving a cone over D_1 and a cone over D_2 whose restrictions to K agree. The representing object for such compatible pairs of cones is the pullback of the two individual limits over the overlap limit. Finally, $a_j^{/S}$ preserves finite limits by Lem. C.2. □

C.3 Validation and Termination

We enforce well-formedness with the validator V (Appendix B). In inference, decoding is constrained by V; violations are rejected before token commitment when they are locally detectable, and completed typed records are checked before acceptance. Termination is enforced operationally by a budget on node blocks/typed records together with the explicit `<summarizer:end>` token.

Defeasible validation. For more complex, non-monotone reasoning, one may choose in advance a finite priority set $\rho \in \{0, \ldots, R\}$ and an increasing chain of Lawvere–Tierney topologies, with associated nuclei $c^{\leq\rho}$. A retraction step from rank ρ to $\rho' < \rho$ replaces the active modality by $c^{\leq\rho'}$ on affected objects. This optional extension lies outside the monotone core; the order-invariance result applies for a fixed active modality.

D Detailed Proofs

This appendix provides detailed proofs for the theorems, propositions, and lemmas presented in the main text. We assume familiarity with basic concepts from category theory and topos theory, as found in references like (MacLane and Moerdijk, 2012; Johnstone, 2002).

D.1 Proofs for Section 3

Proof of Theorem B.1. The proof proceeds by demonstrating totality, determinism, and analyzing the computational complexity.

The syntactic extraction map Φ_{syn} is defined for any trace H that is deemed well-formed by the validator V. Well-formedness ensures that every record in the trace can be unambiguously parsed and assigned a syntactic action.

Every `@node` record has a unique, strictly increasing ID. Every `@edge` record refers to a `src` ID that has already been emitted and a `dst` ID corresponding to the current node, preventing forward

references and cycles in the operational dependency graph of records. Typed records for status, entailments, and equalities have their targets and roles checked for validity.

Since every syntactically valid record has a defined extraction action (e.g., add a node, add an edge, update a state, record a summarizer selection, record an entailment/equivalence), the extraction process is defined for the entire trace. Hence, Φ_{syn} is total on the set of well-formed traces.

We must show that a given well-formed trace H maps to a single, unique syntactic diagram.

The set of nodes V in the extracted DAG G is uniquely determined by the set of `@node` records. The set of operational provenance edges E is uniquely determined by the set of `@edge` records. The selected synthesis presentations are uniquely determined by the accepted `@edge kind=use` records into each summarizer and by the validated states of their source proposer nodes. The index category $\mathcal{J}_G$ is formed by taking the generated category on proposer nodes, certified strengthening/refinement arrows, and certified entailment arrows, then quotienting by the smallest congruence generated by certified equivalence/path-equality records. Operational provenance edges that lack a typed semantic certificate do not enter $\mathcal{J}_G$. The smallest congruence is unique.

After a semantic interpretation $[\![\cdot]\!]$ is fixed, the diagram functor $D_G : \mathcal{J}_G \to (\mathcal{E}/S)$ maps each object (proposer node) to its interpreted object over S and each certified arrow to its interpreted morphism over S.

Because each syntactic step is a deterministic function of the input trace, the final syntactic output $(G, \mathcal{J}_G, \{\mathcal{J}_\Sigma(s)\}_s)$ is unique; with fixed $[\![\cdot]\!]$, the semantic diagram D_G and selected subdiagrams $D_\Sigma(s)$ are unique as well.

Now, let us calculate the complexity:

- Parsing the trace H is a single linear pass, $O(|H|)$.
- Building the graph structure, selected synthesis presentations, and semantic generators involves processing each record once. Using hash maps to store node information (roles, states), this takes $O(|H|)$ time and space, excluding solver calls.
- When only node equivalences are present, managing these with a union-find data structure takes $O(|H|\,\alpha(|H|))$ time, where α is the inverse Ackermann function.
- Solver-backed validation adds the sum of the costs of the selected decision procedures for the typed witnesses.
- When path equalities are introduced, a more complex congruence closure algorithm is needed. While algorithms like e-graphs perform with near-linear amortized time in practice, their worst-case complexity can be higher depending on the specific theory. We state the complexity parametrically in this case.

The stated complexity bounds follow. □

Proof of Theorem B.3. The core insight is that the extraction map Φ is sensitive only to the set of normalized typed records and their dependency partial order, not the specific linear sequence in which they appear, provided that sequence is a valid topological sort of the dependency graph.

Since H_1 and H_2 contain the same multiset of normalized typed records, they will result in the same set of proposer nodes, the same dependency edges between nodes, the same status assignments, the same selected summarizer sources, the same set of entailment records, and the same set of declared equivalences. The normalization step resolves implicit fields such as omitted witnesses, so equality is being tested on explicit records rather than on surface syntax.

The well-formedness rules (`src` < `dst`, etc.) ensure that any valid trace is a topological sort of the underlying operational dependency graph of records. If H_1 and H_2 induce the same dependency partial order, they are simply two different valid topological sorts of the same abstract structure.

The extraction map Φ constructs the diagram by first identifying all nodes, provenance edges, summarizer selections, and semantic generators from the records and then applying the validated equalities. This process does not depend on the linear order of emission, only on the final set of normalized records and their dependency constraints. Therefore, $\Phi(H_1)$ and $\Phi(H_2)$ produce isomorphic abstract graphs, the same validated equalities, and thus isomorphic index categories $\mathcal{J}_G$ and selected presentations $\mathcal{J}_\Sigma$. By Proposition 4.10, isomorphic diagrams have isomorphic limits. After reflection, the resulting summaries are isomorphic. In the posetal sub-case where the summary is a specific subobject (the meet-plus-closure), the summaries are equal. □

D.2 Proofs for Section 4

Theorem D.1 (DoT Process as Diagram Construction)**.** A DoT reasoning process generating a well-formed typed DAG $G = (V, E)$, together with a fixed semantic interpretation of typed propositions and certified arrows, defines a functor (a diagram) $D_G : \mathcal{J}_G \to (\mathcal{E}/S)$ with:

- Each typed proposer node v mapped to an object $D_G(v) \to S$ in the slice (a subobject $D_G(v) \hookrightarrow S$ in the predicate/mono fragment);
- Accepted critic records supply certified arrows, isomorphisms, or path equalities between proposer objects; critic nodes are witnesses for these records rather than objects of D_G;
- Each arrow $v \to u$ in $\mathcal{J}_G$ is mapped to a certified morphism over S, witnessing entailment or strengthening coherence (posetal case: an inclusion $D_G(v) \hookrightarrow D_G(u)$).

Functoriality ensures that composite certified dependencies (via paths modulo $\equiv$) are respected in the slice.

Proof. We construct the functor $D_G : \mathcal{J}_G \to (\mathcal{E}/S)$ by defining its action on objects and morphisms and verifying that it satisfies the functoriality axioms.

As per Definition 4.3, the extraction map Φ applied to the trace yields a DAG G and an index category $\mathcal{J}_G$ with proposer nodes as objects and syntactically certified arrows as morphism generators, modulo certified equality constraints.

For each object $v \in \mathrm{Ob}(\mathcal{J}_G)$, Assumption (A1) states that its content can be interpreted as an object over S (a subobject of S in the predicate/mono fragment). We define the action of D_G on objects as this interpretation:

$$D_G(v) := [\![\mathrm{content}(v)]\!] \to S.$$

This defines an object in the slice category $(\mathcal{E}/S)$.

Now let $\alpha : v \to u$ be a generator in $\mathcal{J}_G$. If α is induced by a certified strengthening/refinement record, Assumption (A1) assigns it a morphism $D_G(v) \to D_G(u)$ over S. If α is induced by `@entails src=v dst=u`, the certified entailment directly supplies the same kind of morphism. If α is part of a certified `@eq` record, then in the predicate fragment it is mutual entailment, while in the general slice fragment the witness supplies an isomorphism and inverse equations. We then extend D_G from generators to arbitrary morphisms by composition.

Well-definedness with respect to $\equiv$ follows from Assumption (A3): if two generated composites are identified by validated equality records, then their interpreted composites in the slice are equal. Identities map to identities, and composition in $\mathcal{J}_G$ maps to composition in $(\mathcal{E}/S)$. Hence D_G is a functor. Thus, D_G is a valid functor from the index category $\mathcal{J}_G$ to the slice category $(\mathcal{E}/S)$. □

Theorem D.2 (Summarization as finite meet-plus-closure in the reflective subposet)**.** Assume every proposer selected for synthesis is interpreted as a c-closed subobject of S and every certified semantic arrow between selected proposers is an inclusion, so the selected diagram D_Σ lands in the thin category $\mathrm{Sub}(S)$. Let V_Σ be the finite set of selected validated proposer nodes. Then the finite-limit summary is

$$\mathrm{Summary}_\Sigma = \lim_{\mathrm{Sub}(S)} D_\Sigma = \bigwedge_{v \in V_\Sigma} D_G(v),$$

equivalently the finite colimit of D_Σ^{op} in $\mathrm{Pred}(S) = \mathrm{Sub}(S)^{\mathrm{op}}$. The reflected form

$$c\Big(\bigwedge_{v \in V_\Sigma} D_G(v) \Big)$$

is equal to this meet.

Proof. Work in $\mathrm{Sub}(S)$ ordered by inclusion, and in the opposite poset $\mathrm{Pred}(S) = \mathrm{Sub}(S)^{\mathrm{op}}$ (information order). For a finite family $\{P_v\}$, the join (colimit) in $\mathrm{Pred}(S)$ corresponds to the meet (intersection) in $\mathrm{Sub}(S)$:

$$\bigvee\nolimits_{\mathrm{Pred}(S)} P_v \;=\; \bigwedge\nolimits_{\mathrm{Sub}(S)} P_v.$$

More generally, since $\mathrm{Sub}(S)$ is thin, the finite limit of the selected posetal diagram is the greatest lower bound of its objects. Equivalently, after passing to the opposite diagram $D_\Sigma^{\mathrm{op}} : \mathcal{J}_\Sigma^{\mathrm{op}} \to \mathrm{Pred}(S)$, this same object is the information-order colimit. Now assume each validated proposition is c-closed, i.e. $c(P_v) = P_v$, and that c is a nucleus (finite-meet preserving; Assumption 4.2). Then the meet of validated propositions is again c-closed:

$$c\Big(\bigwedge_v P_v \Big) \;=\; \bigwedge_v c(P_v) \;=\; \bigwedge_v P_v.$$

Thus, the synthesis object lies in the c-closed fragment and agrees with the information-colimit. We may write the summary as $c(\bigwedge_v P_v)$ to match the general-arrow presentation, noting that under the standing assumptions the outer c is redundant. □

Corollary D.3 (General case via slice sheafification)**.** For a general selected diagram $D_\Sigma : \mathcal{J}_\Sigma \to (\mathcal{E}/S)$ with non-posetal arrows and certified coherence/path-equality relations already satisfied by the extracted functor, the synthesized summary is

$$\mathrm{Summary}_\Sigma \;\cong\; \lim_{\mathrm{Sh}_j(\mathcal{E})/a_j S} \big(a_j^{/S} \circ D_\Sigma\big) \;\cong\; a_j^{/S}\big(\lim_{\mathcal{E}/S} D_\Sigma\big),$$

where $a_j^{/S} : (\mathcal{E}/S) \to (\mathrm{Sh}_j(\mathcal{E})/a_j S)$ is the functor induced by sheafification.

Proof. The logic is analogous to the posetal case but lifted from posets to general categories, using finite limits rather than meets.

In the presheaf topos setting, the slice $(\mathcal{E}/S)$ is finitely complete. Since selected diagrams from finite runs have finite graph presentations, the required limit $L = \lim D_\Sigma$ is a finite-limit construction in $(\mathcal{E}/S)$: finite products collect cone components, and equalizers impose compatibility with the finitely many generating arrows. Certified path-equality records are checked when constructing the functor D_Σ; they are not used to repair an otherwise incoherent diagram at synthesis time. Intuitively, this limit enforces simultaneous satisfaction/compatibility of the selected validated constraints represented by the diagram.

The Lawvere–Tierney topology j defines a reflective subcategory of j-sheaves, $\mathrm{Sh}_j(\mathcal{E}) \hookrightarrow \mathcal{E}$. The reflector is the sheafification functor a_j. This induces a left exact functor on slice categories, $a_j^{/S} : (\mathcal{E}/S) \to (\mathrm{Sh}_j(\mathcal{E})/a_j S)$, as stated in Lemma C.2. This functor maps objects in the slice to their validated/sheaf counterparts over $a_j S$.

Since $a_j^{/S}$ is left exact, it preserves finite limits. Therefore, the summary (synthesis computed within the validated/sheaf slice) can be obtained by computing the finite limit in the ambient slice and then applying the reflector:

$$\mathrm{Summary}_\Sigma \cong \lim_{\mathrm{Sh}_j(\mathcal{E})/a_j S} (a_j^{/S} \circ D_\Sigma) \cong a_j^{/S}(\lim_{(\mathcal{E}/S)} D_\Sigma).$$

When restricted to subobjects, the sheafified mono lives over $a_j S$. Pulling it back along $\eta_S : S \to a_j S$ gives the j-closure c_S of the original subobject. The colimit in the information order corresponds to the meet in inclusion order after reversing variance. Thus, for a posetal diagram, comparison back over S gives $c_S(\bigwedge_v D_G(v))$, recovering the result of Theorem 4.5. □

Theorem D.4 (Conditional consistency via closure validation)**.** Fix $\mathcal{E} = \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$ and a closure c on $\mathrm{Sub}(S)$. In the fixed-stage finite-family setting, if the selected validated family is jointly satisfiable relative to a fixed stage $a_0 \in \mathcal{C}$, then the summary is non-initial. In the model-compact setting, finite satisfiability of the corresponding first-order family implies satisfiability in some model, but not necessarily in the intended/standard model and not necessarily as componentwise non-emptiness in a preassigned presheaf stage.

Proof. Let $L = \bigwedge_{v \in V_\Sigma} D_G(v)$ be the meet of the interpretations of the selected validated propositions. In the fixed-stage case, the summary is $\mathrm{Summary}_\Sigma = c(L)$. We show that the summary is a non-initial subobject of S.

The premise states that the finite family $\{D_G(v)\}_{v \in V_\Sigma}$ is jointly satisfiable relative to a fixed stage $a_0 \in \mathcal{C}$. In the presheaf topos $\mathcal{E} = \mathrm{Set}^{\mathcal{C}^{\mathrm{op}}}$, this means there exists an element $x \in S(a_0)$ such that for every $v \in V_\Sigma$, x lies in the subset $D_G(v)(a_0) \subseteq S(a_0)$.

The existence of such an element x implies that the intersection of these subsets is non-empty:

$$\bigcap_{v \in V_\Sigma} D_G(v)(a_0) \neq \emptyset.$$

In a presheaf topos, finite limits are computed componentwise. Therefore, the component of the meet subobject L at stage a_0 is precisely this intersection:

$$L(a_0) = \left(\bigwedge_{v \in V_\Sigma} D_G(v) \right)(a_0) = \bigcap_{v \in V_\Sigma} D_G(v)(a_0).$$

Since this set is non-empty, the subobject L is non-initial.

The closure operator c is extensive, meaning that for any subobject P, we have an inclusion $P \hookrightarrow c(P)$. Since presheaf monomorphisms are componentwise injections, applying this to our meet L at stage a_0 gives:

$$L(a_0) \subseteq (c(L))(a_0).$$

Since we established that $L(a_0)$ is non-empty, its superset $(c(L))(a_0)$ must also be non-empty. The summary, $\mathrm{Summary}_\Sigma = c(L)$, has a non-empty component at stage a_0. Therefore, the summary is a non-initial subobject.

For the model-compact setting, compactness is applied to the first-order family represented by the typed propositions. If every finite subset is satisfiable together with the background theory, then there is a model and assignment satisfying the entire family. When S and c are interpreted in that same model, extensivity again preserves satisfiability after closure. This proves consistency in the model-theoretic sense while deliberately avoiding any intended-model or fixed-stage presheaf claim. □

Proposition D.5 (Robustness under Diagram Isomorphisms)**.** Let $D_{\Sigma,1} : \mathcal{J}_{\Sigma,1} \to (\mathcal{E}/S)$ and $D_{\Sigma,2} : \mathcal{J}_{\Sigma,2} \to (\mathcal{E}/S)$ represent selected validated reasoning steps from two different runs. If there is an isomorphism of diagrams, then their slice-limits are isomorphic.

Proof. Let (σ, η) be an isomorphism of diagrams, i.e. $\sigma : \mathcal{J}_1 \to \mathcal{J}_2$ is an isomorphism of index categories and $\eta : D_1 \Rightarrow D_2 \circ \sigma$ is a natural isomorphism. Precomposition with σ induces an equivalence between cones over D_2 and cones over $D_2 \circ \sigma$; moreover, the natural isomorphism η induces an equivalence between cones over D_1 and cones over $D_2 \circ \sigma$. Equivalences preserve terminal objects, so the terminal cone over D_1 (its limit) corresponds to the terminal cone over D_2 (its limit). Hence $\lim D_1 \cong \lim D_2$. □

Proposition D.6 (Properties of Synthesis)**.** The synthesis operation, $\text{Summary}(\mathcal{P}) = c(\bigwedge_{P \in \mathcal{P}} P)$, exhibits Monotonicity, Idempotence, and Conservativity.

Proof. Let $\mathcal{P} = \{P_v\}_{v \in V_\Sigma}$. The properties follow directly from the definition of the summary and the standard properties of meet ($\wedge$) in a poset and a closure operator (c).

For monotonicity, we want to show that $\text{Summary}(\mathcal{P} \cup \{P_{\text{new}}\}) \subseteq \text{Summary}(\mathcal{P})$. The meet of a larger set of subobjects is always a subobject of the meet of a smaller set:

$$\bigwedge_{P \in \mathcal{P} \cup \{P_{\text{new}}\}} P = \left(\bigwedge_{P \in \mathcal{P}} P \right) \wedge P_{\text{new}} \subseteq \bigwedge_{P \in \mathcal{P}} P.$$

The closure operator c is monotone: if $X \subseteq Y$, then $c(X) \subseteq c(Y)$. Applying c to both sides of the inclusion above preserves the relation:

$$c\left(\bigwedge_{P \in \mathcal{P} \cup \{P_{\text{new}}\}} P \right) \subseteq c\left(\bigwedge_{P \in \mathcal{P}} P \right).$$

This is the desired result.

For idempotence, we want to show $\text{Summary}(\mathcal{P}) = c(\bigwedge_{P \in \mathcal{P}} c(P))$. Using finite-meet preservation of c,

$$c\left(\bigwedge_{P \in \mathcal{P}} c(P) \right) = \bigwedge_{P \in \mathcal{P}} c(c(P)) = \bigwedge_{P \in \mathcal{P}} c(P).$$

If all propositions in $\mathcal{P}$ are validated, then each is already c-closed, so $P = c(P)$, and the displayed expression equals $c(\bigwedge_{P \in \mathcal{P}} P)$.

For conservativity, assume $P_w \supseteq \bigwedge_{P \in \mathcal{P} \setminus \{P_w\}} P$. We want to show $\text{Summary}(\mathcal{P}) = \text{Summary}(\mathcal{P} \setminus \{P_w\})$. The meet of all propositions in $\mathcal{P}$ is:

$$\bigwedge_{P \in \mathcal{P}} P = \left(\bigwedge_{P \in \mathcal{P} \setminus \{P_w\}} P \right) \wedge P_w.$$

Since P_w contains the meet of the others, intersecting with P_w does not change the result. That is, if $X \subseteq Y$, then $X \wedge Y = X$. Therefore:

$$\left(\bigwedge_{P \in \mathcal{P} \setminus \{P_w\}} P \right) \wedge P_w = \bigwedge_{P \in \mathcal{P} \setminus \{P_w\}} P.$$

Applying the closure operator c to both sides gives:

$$c\left(\bigwedge_{P \in \mathcal{P}} P \right) = c\left(\bigwedge_{P \in \mathcal{P} \setminus \{P_w\}} P \right),$$

which is precisely $\text{Summary}(\mathcal{P}) = \text{Summary}(\mathcal{P} \setminus \{P_w\})$. □